# Hockey UQ Project — Architecture & Build Plan

A probabilistic hockey prediction pipeline with full uncertainty quantification, structured as a port and extension of the AEM 8290 land-surface column UQ project. The goal is calibrated predictive distributions over game outcomes, player point totals, and (eventually) season-long quantities — not point estimates.

This document is the project's reference architecture and roadmap. Come back to it when you forget what `analysis/` is supposed to contain or why `game_model.py` lives at the top level of `models/`.

---

## Project Goals

The end-state pipeline produces, for any matchup:

- $P(\text{home win})$ with credible intervals
- Predictive distributions over final score, total goals, goal differential
- Predictive distributions over individual player point totals
- Sensitivity decomposition: which inputs drive outcome variance

All of these come from the same underlying machinery — a hierarchical Bayesian model fit to league-wide data, propagated through a game model and Monte Carlo simulator, validated with proper scoring rules.

**First shipping target:** single-game win probability. Smallest pipeline that exercises every layer.

---

## Status

*Last updated: 2026-10-24*

| Phase | Status | Result |
|---|---|---|
| 1. Data foundation | **Done** | 5 seasons (2021-22 to 2025-26), 6,560 games, full play-by-play, rebound/rush context validated |
| 2. xG model | **Done** | LightGBM, test AUC 0.769, team-season xG correlates 0.97 with MoneyPuck |
| 3. Team strength & goalie aggregates | **Done** | Rolling 25-game xG team strength picks winners 59% of the time; Bayesian goalie rating (k = 800, season-adjusted) |
| 4. Track A toy MCMC | **Done** | AM sampler verified: 90% intervals cover the truth 90 and 96 times out of 100; matches the exact posterior; COL vs DAL fit saved |
| 5. Track B hierarchical model | **Done** | Full-league PyMC model (2024-25 + 2025-26, 2,624 games): team offense/defense ratings with uncertainty, home ice 0.071, tie inflation ρ = 0.51; synthetic recovery, Track A match and posterior predictive checks pass; remaining dispersion gap traced to empty-net removal (section 11) |
| 6. Game model + Monte Carlo | **Done** | `predict_game.py`: P(win) with uncertainty from an exact per-draw model (reproduces home win, REG/OT/SO shares of 2024-26); score outputs from a 10-second score-state simulator (pulled goalies, late-game caution) that matches real ties, empty-net share and margins |
| 7. Validation | **Done** | Walk-forward backtest, 2,624 held-out games (2024-25 + 2025-26). Track B v1.1 (ratings from xG + goals, 1-year window) log loss 0.6773, calibration slope 0.99: clearly beats Track B v1, Elo and home rate; **statistically tied with rolling xG** (0.6754). Recency weighting tested and not adopted. Market comparison waiting on closing-line data |
| 8. PCE / Sobol | **Done** | What makes P(home win) differ between games: team ratings 95%, home-ice uncertainty 3%, starting goalies 2%, almost no interactions. Ties are driven by tie-inflation uncertainty (55%). PCE and Saltelli agree within 0.0003. Analysis B (one matchup's uncertainty) moved to v2 |

| Phase | Status | Result |
|---|---|---|
| v2. Model improvements | Planned | Top item from Phase 7: time-varying team ratings. Then hierarchical xG layer, lineup (player-level) model; see *v2 — Planned Improvements* |

**Check current state:** `py .\scripts\check_state.py` lists which files and tables exist and whether `shot_context` matches the current derivation code.

---

## The Six-Step Arc

The full pipeline, end-to-end:

1. **Build component models** — xG (gradient boosting), team strength aggregates, goalie ratings. Each produces estimates with uncertainty.
2. **Combine via Poisson/NB game model** — maps team-level inputs to $\lambda_{\text{home}}, \lambda_{\text{away}}$.
3. **Monte Carlo simulation** — propagates uncertainty from every layer into a predictive distribution over outcomes.
4. **Aggregate sims** into the distributions you actually care about: win probabilities, score distributions, player point totals.
5. **Validate** with proper scoring rules (log-loss, Brier, CRPS) and calibration plots on held-out games.
6. **Fit PCE surrogate** over the simulator's input space — enables fast sensitivity analysis and analytical Sobol decomposition.

---

## Mapping to the AEM 8290 Soil-Atmosphere Project

The hockey UQ pipeline is structurally the same as your AEM 8290 land-surface column project, with the forward model swapped and a hierarchical layer added. Most of the conceptual machinery already exists in working code.

| Hockey project component | Soil project equivalent |
|---|---|
| Component models (xG, team strength, goalie) | Forward model: soil heat + Richards' + SEB |
| Game model ($\lambda_{\text{home}}, \lambda_{\text{away}}$) | Forward model mapping $\theta \to$ QoIs |
| Monte Carlo over game outcomes | Forward propagation, screening, Sobol Monte Carlo |
| Aggregation to predictive distributions | Posterior moments, predictive distributions |
| Held-out validation, calibration plots | LOO-CV $Q^2$, held-out RMSE, trace plots, ACF |
| PCE surrogate for Sobol indices | PCE surrogate for MCMC efficiency |

**What transfers directly:**

- The Adaptive Metropolis sampler (random-walk MH, burn-in / adapt / sample, $(2.38)^2/d$ scaling, $\xi \in [-1, 1]^d$ standardized space)
- Additive Gaussian likelihood structure (mechanics — independent QoI noise summed in log-space)
- Objective prior strategy (uniform-bounded for location, log-uniform for scale spanning decades)
- PCE-as-surrogate trick for expensive forward models
- Validation discipline ($Q^2$, trace plots, acceptance rates, ACF, posterior recovery vs truth)

**What changes:**

- **Dimensionality.** Soil: 5 calibration parameters. Hockey hierarchical model: ~70–100+ parameters. Random-walk MH does not scale; NUTS (HMC variant via PyMC/NumPyro) required.
- **Hierarchy.** Soil model is flat. Hockey model has population-level hyperpriors with partial pooling across teams and positions.
- **Likelihood form.** Gaussian becomes Poisson (goal counts) or Bernoulli (win/loss).
- **Forward model speed.** Soil: ~1 s/eval, surrogate essential. Hockey game model: milliseconds/eval, surrogate optional. PCE earns its keep in hockey for the **Sobol analysis**, not for inference speed.
- **Data volume.** Soil: synthetic 24–72h time series of 2 QoIs. Hockey: ~1200 games/season × multiple observables per game.

---

## Architecture

## Folder Structure

```
hockey-stats/
│
├── pyproject.toml
├── README.md
├── .gitignore
├── .env.example
│
├── config/
│   ├── settings.py             # paths, current season, DB location
│   └── teams.py                # team metadata, colors, abbreviations
│
├── data/                       # SQLite database lives here
│   ├── hockey_stats.db
│   └── external/               # files downloaded by hand (e.g. MoneyPuck teams.csv)
│
├── src/
│   │
│   ├── data/                   # data acquisition & storage
│   │   ├── __init__.py
│   │   ├── nhl_api.py          # NHL API client
│   │   ├── db.py               # SQLite operations
│   │   ├── scraper.py          # Natural Stat Trick / Evolving-Hockey scrapers
│   │   └── cache.py            # caching layer
│   │
│   ├── processing/             # raw data → analysis-ready
│   │   ├── __init__.py
│   │   ├── shot_context.py     # DONE: rebound / rush flags → shot_context table
│   │   ├── xg_processing.py    # DONE: xG feature table, one row per shot
│   │   ├── shots.py            # shot data processing
│   │   ├── corsi.py            # Corsi/Fenwick calculations
│   │   ├── game_score.py       # game score breakdown
│   │   ├── team_strength.py    # DONE: team_game_xg, rolling_strength, strength_as_of
│   │   ├── stints.py           # v2: shifts → stints (constant on-ice sets)
│   │   ├── rapm.py             # RAPM / WAR (future)
│   │   ├── goalie.py           # DONE: goalie_game_xg (per-goalie, per-game xGA / GA)
│   │   ├── game_table.py       # DONE: one row per game for Track B (goals, starters, goalie γ)
│   │   ├── late_game.py        # DONE: OT factor, shootout, empty-net table (late_game.json)
│   │   └── score_state.py      # DONE: score effects, pull timing, 6-on-5 / empty-net rates
(score_state.json)
│   │
│   ├── models/
│   │   ├── __init__.py
│   │   ├── schemas.py          # Pydantic: Game, Player, Shot, ScheduleGame
│   │   ├── game_model.py       # DONE: exact outcome probabilities per posterior draw
│   │   ├── active_fit.py       # DONE: which saved fit the live pipeline uses (active_fit.json)
│   │   │
│   │   ├── deterministic/
│   │   │   ├── __init__.py
│   │   │   ├── xg.py           # DONE: ExpectedGoalsModel (LightGBM)
│   │   │   └── rapm.py         # ridge regression RAPM (future)
│   │   │
│   │   └── stochastic/
│   │       ├── __init__.py
│   │       ├── am_sampler.py   # Adaptive Metropolis (ported from soil)
│   │       ├── goalie.py       # DONE: goalie_rating_as_of (Gamma-Poisson, season-adjusted)
│   │       ├── toy_mcmc.py     # Track A: hockey-flavored AM MCMC
│   │       ├── hierarchical.py # DONE: Track B v1 PyMC model, sample_scores, simulate_league
│   │       ├── hierarchical_xg.py # DONE: Track B v1.1 (ratings from xG + goals),
simulate_league_xg
│   │       ├── xg_hierarchical.py # v2: Bayesian layer on top of LightGBM xG
```

```
│   │       ├── lineup_rapm.py  # v2: Bayesian RAPM player effects
│   │       ├── montecarlo.py   # DONE: tie-inflated and score-state game simulators
│   │       └── pce.py          # DONE: PCE (ported from soil), Sobol' from coefficients, Saltelli,
Ishigami test
│   │
│   ├── analysis/               # NEW: reduce sim output to distributions
│   │   ├── __init__.py
│   │   ├── aggregation.py      # DONE: outcome tables, likely scores, totals, puck lines, fair
odds
│   │   └── sensitivity.py      # DONE: Phase 8 inputs/ranges, vectorized exact game model, group
indices
│   │
│   ├── validation/             # scoring, calibration, backtesting
│   │   ├── __init__.py
│   │   ├── scoring.py          # DONE: log loss, Brier, calibration slope/table, paired bootstrap,
RPS for totals
│   │   ├── baselines.py        # DONE: home rate, tuned Elo, rolling-xG logistic
│   │   └── backtest.py         # DONE: walk-forward harness (weekly refits, resumable)
│   │
│   ├── pipelines/              # NEW: high-level orchestration
│   │   ├── __init__.py
│   │   ├── train_xg.py         # DONE: build features, train, evaluate, save
│   │   ├── score_xg.py         # DONE: score every shot into shot_xg (leave-one-season-out)
│   │   ├── fit_hierarchical.py # DONE: fit Track B (default v1.1, last 365 days), save NetCDF +
ratings, set active fit
│   │   ├── fit_toy.py          # fit Track A toy MCMC for comparison
│   │   ├── fit_xg_layer.py     # v2: fit hierarchical xG layer, save NetCDF
│   │   ├── fit_lineup.py       # v2: build stints, fit RAPM, save NetCDF
│   │   ├── predict_game.py     # DONE: py -m src.pipelines.predict_game HOME AWAY [--date ...]
│   │   ├── backtest_season.py  # DONE: py -m src.pipelines.backtest_season --season S [--model xg]
[--window D] [--half-life D]
│   │   ├── run_baselines.py    # DONE: py -m src.pipelines.run_baselines --seasons S1 S2
│   │   ├── predict_day.py      # DONE: forward test, log every game on a date before it's played
│   │   ├── score_forward.py    # DONE: score the forward-test log against played games
│   │   ├── make_cards.py       # DONE: pregame cards (called by predict_day) and postgame cards (-
-pending, --date, ids)
│   │   └── build_pce.py        # DONE: py -m src.pipelines.build_pce [--degree 4]
│   │
│   ├── viz/                    # visualization (existing)
│   │   ├── __init__.py
│   │   ├── theme.py
│   │   ├── rink_plot.py        # DONE: draw_rink, plot_game_events, plot_game
│   │   ├── card_style.py       # DONE: card theme, team-colour pairing (colour-blind checked),
odds/pct helpers
│   │   ├── pregame_card.py     # DONE: P(win) posterior strip, fair odds, how the game ends, score
grid, totals
│   │   ├── postgame_card.py    # DONE: shot heat map, deserve-to-win meter, xG worm, model vs
result, shooters, goalies
│   │   ├── shot_heatmap.py
│   │   ├── game_summary.py     # (future)
│   │   ├── player_card.py      # (future)
│   │   └── components/
│   │
│   └── export/
│       ├── __init__.py
│       └── save.py
│
├── notebooks/                  # exploration and checks
│   ├── data_sanity.ipynb       # raw data checks (sections 1-13)
│   ├── xg_features_check.ipynb # xG feature table checks
│   ├── xg_model_dev.ipynb      # baseline, LightGBM, evaluation, MoneyPuck comparison
│   ├── phase3_check.ipynb      # team strength + goalie rating checks and settings
│   ├── phase4_toy.ipynb        # Track A function checks + recovery/coverage tests
│   ├── phase5_check.ipynb      # Track B checks (recovery, Track A match, predictive)
```

```
│   ├── phase6_check.ipynb      # game model checks (exact vs simulated, history, calibration
preview)
│   ├── phase7_check.ipynb      # backtest scoring: models vs baselines, paired tests, calibration,
blends, market
│   └── phase8_check.ipynb      # PCE/Sobol checks: Ishigami, degree, accuracy, PCE vs Saltelli,
groups, robustness
│
├── output/                     # generated images, posterior samples
│   ├── game_summaries/
│   ├── player_cards/
│   ├── posteriors/             # NetCDF posterior samples
│   └── models/                 # pickled trained models (xG); keep out of git
│
├── tests/
│   ├── test_corsi.py
│   ├── test_game_score.py
│   └── fixtures/
│
└── scripts/                    # CLI entry points
    ├── load_all.py                   # teams, rosters, schedule, games, shots, events, shifts;
seasons from SEASONS in config/settings.py; 1 s between requests, retries on 429, skips complete
schedules
    ├── backfill_game_events.py       # one-time: full play-by-play for loaded games
    ├── rebuild_shot_context.py       # re-derive shot_context after a code change
    ├── check_state.py                # project state + shot_context sync check
    ├── fix_goal_shooters.py          # one-time repair (goal scorer IDs)
    ├── check_block_attribution.py    # diagnostic: who owns a blocked-shot event
    ├── check_block_zone_frame.py     # diagnostic: which frame blocked-shot zones use
    ├── nst_spotcheck.py              # compare rush/rebound counts with Natural Stat Trick
    └── rush_event_audit.py           # list rush-flagged shots for one game
```

### Why Certain Things Live Where They Do

`models/game_model.py` **at the top level of** `models/` — neither purely deterministic nor purely stochastic. It's the bridge that takes deterministic components and lifts them into a stochastic combiner. Forcing it into one subfolder would be a category error. It belongs alongside `schemas.py` as a top-level model concern.

`analysis/` **separate from** `models/` — `models/` is about computation that produces parameters or predictions. `analysis/` is about *reducing* simulation output to the distributions you care about. Different responsibility: aggregation, not modeling.

`validation/` **separate from everything** — scoring rules and calibration plots are infrastructure that applies to any model. Keeping them separate means they can be reused if you swap component models.

`pipelines/` **for orchestration** — high-level entry points that chain components together. Each pipeline file should read like a recipe: load this, fit that, save the output. No business logic, just glue.

**Two tracks under** `stochastic/` — `toy_mcmc.py` (Track A, your soil-pipeline port) and `hierarchical.py` (Track B, PyMC) coexist. Track A is the reference implementation; Track B is the production model. They share input contracts so the rest of the pipeline doesn't care which produced the samples.

---

## Data Contract

### Tables

| Table | Rows | One row per | Notes |
|---|---|---|---|
| `shot_events` | 773,259 | shot attempt (incl. blocked, shootout) | Raw; filter at query time (below) |
| `game_events` | 2,060,996 | play-by-play event of any type | Raw; ordered by `sort_order`, never `event_id` |

| Table | Rows | One row per | Notes |
|---|---|---|---|
| `shot_context` | 566,545 | unblocked, non-shootout shot | Derived from `game_events` by `shot_context.py` |
| `shot_xg` | 562,374 | scored shot (unblocked, periods 1–4, goalie in net) | Derived by `score_xg.py`; each season scored by a model that did not train on it |
| `shifts` | 4,578,542 | player shift | |
| `games`, `schedule` | 6,560 | game | 1,312 per season × 5 |

`shot_context` joins to `shot_events` on `(game_id, event_id)`. After any change to `src/processing/shot_context.py`, run `scripts/rebuild_shot_context.py`; `check_state.py` flags a stale table.

### Filtering Rules

`shot_events` is stored **raw and complete** — no filtering is applied at the database level. Shootout attempts (period 5) and blocked shots are all present. This is deliberate: the raw table stays a faithful record, and every consumer applies the filters it needs at query time.

**Two rules, applied at query time:**

1. `period <= 4` — excludes shootout (period 5) events; keeps OT (period 4).
2. `event_type IN ('shot-on-goal', 'goal', 'missed-shot')` — excludes blocked shots (degenerate coordinates) and shootout attempts.

Together these yield **566,545 clean unblocked non-shootout shots**. The xG feature table then drops empty-net shots (opposing goalie digit = 0 in `situation_code`), leaving **562,374 training shots** (Phase 2).

**Why this matters per consumer:**

- **xG model (Phase 2):** apply both rules, then drop empty-net shots. Blocked shots lack a true shot location, shootout attempts are mislabeled as goals, and empty-net shots score regardless of geometry; any of them would poison training.
- **Team model (Phase 5) and game model (Phase 6):** fit **regulation goals** (`period <= 3`, empty-net goals excluded). Overtime is sudden death, which a per-game scoring rate can't describe, so Phase 6 simulates OT and the shootout separately after a tied regulation. (Phase 4's Track A used `period <= 4` goals; the matched check uses the same.)

**Background:** the data-sanity notebook found that period-5 shootout attempts are stored as `event_type 'goal'` / `'shot-on-goal'` / `'missed-shot'` (1,065 shootout "goals", ~2.6% of all goal events). All score-reconciliation residuals after filtering are off-by-exactly-1 (the shootout-winner credit added to the official score with no corresponding real goal event), zero off-by-more-than-1 — fully characterized and expected, not a bug.

### Known Data Quirks (verified against this database)

- **Goal scorers:** the API stores the scorer under `scoringPlayerId`, not `shootingPlayerId`. `parse_shot_events` reads both. 41,536 goals were repaired once with `scripts/fix_goal_shooters.py`.
- **Blocked-shot owner:** `eventOwnerTeamId` is the **shooting** team (90/10, `check_block_attribution.py`).
- **Blocked-shot zone:** `zoneCode` is written from the **defending** team's side even though the owner is the shooter (99.9% of team-periods, `check_block_zone_frame.py`). `shot_context.py` flips it before the rush rule.
- **Zone codes** are relative to the event owner (O / N / D). Compare two events only after translating both into the shooter's frame.
- `situation_code` is 4 digits: away goalie, away skaters, home skaters, home goalie (`1551` = 5v5). Decode from the shooter's point of view.
- **Shot-type labels shifted in 2025-26** (many wrist → snap, deflected → tip-in). The xG features group them as `wrist-snap` and `redirect`.
- **Missing shot type** occurs almost only on goals; filling it with "unknown" leaks the outcome. The xG features fill it with "wrist".
- **Coordinates:** center ice is (0, 0), goal lines at x = ±89, blue lines at x = ±25. Teams switch ends each period; normalize per game, period and team.

### Rebound and Rush Definitions (`shot_context.py`)

NST conventions, with the no-stoppage rule enforced (faceoffs, whistles, penalties and goals break the chain):

- **Rebound:** unblocked shot within 3 s of a prior attempt by the same team. 9.4% of shots; converts at 12.4% vs 6.6% for other shots.
- **Rush:** unblocked shot within 4 s of any event in the neutral zone or the shooter's defensive zone. 2.7% of shots; weak as a yes/no flag (7.4% vs 7.1%), but strong when split by the previous event: takeaway 10.1%, blocked shot 11.1%, giveaway 9.0%, hit 4.0%.
- NST spot-check: 1 of 3 rush flags matched in game 2025020993. Both misses had an opponent's shot as the precursor, at physically impossible 1–2 s gaps (about 17% of rush flags, 0.5% of training data).

---

## Parallel Build: Track A vs Track B

The honest version of the model is the PyMC hierarchical NUTS sampler. But porting your AEM 8290 AM MCMC structure to a tiny hockey problem first buys real value: a reference implementation you trust, and an independent cross-check on the PyMC model.

### Track A — Toy AM MCMC (Soil Pipeline Port)

**Scope:** one team pair at a time. Parameter vector:

$$\theta = (\mu_{\text{off,A}}, \mu_{\text{def,A}}, \mu_{\text{off,B}}, \mu_{\text{def,B}})$$

**Likelihood:** Poisson on observed goal counts in head-to-head games:

$$p(y^{\text{obs}} \mid \theta) = \prod_{g} \text{Poisson}(G_{\text{home},g} \mid \lambda_{\text{home}}(\theta)) \cdot \text{Poisson}(G_{\text{away},g} \mid \lambda_{\text{away}}(\theta))$$

**Priors:** bounded uniform in standardized $\xi \in [-1, 1]^4$ space, same as soil.

**Identifiability — only two of the four parameters are learnable.** With $\lambda_A = \exp(\alpha_0 + \mu_{\text{off},A} - \mu_{\text{def},B})$ and $\lambda_B = \exp(\alpha_0 + \mu_{\text{off},B} - \mu_{\text{def},A})$, adding the same constant to $\mu_{\text{off},A}$ and $\mu_{\text{def},B}$ (or to $\mu_{\text{off},B}$ and $\mu_{\text{def},A}$) leaves the likelihood unchanged. The data pin down only the two differences

$$d_A = \mu_{\text{off},A} - \mu_{\text{def},B}, \qquad d_B = \mu_{\text{off},B} - \mu_{\text{def},A}$$

and the rest of the posterior is the bounded prior. Sample $(d_A, d_B)$ directly ($d = 2$).

**Fixed league terms.** Both come from the whole league, computed at fit time by `league_baseline()`, not estimated from the pair:

$$\log \lambda_A = \alpha_0 + \eta \cdot \text{home}_A + d_A, \qquad \log \lambda_B = \alpha_0 + \eta \cdot \text{home}_B + d_B$$

- $\alpha_0 = \log(\text{league away goals per game})$ = 1.0291 (2.799 goals)
- $\eta = \log(\text{home rate}/\text{away rate})$ = 0.0715 (3.006 vs 2.799 goals)

One pair's ~15 games would give $\eta$ an uncertainty of about ±0.20, three times the effect; the league gives about ±0.010. Fixing it uses the precise estimate, keeps the model at two parameters, and avoids $\eta$ trading off against $d_A, d_B$. The ignored ±0.010 is tiny next to the ~±0.15 width of each $d$. (A tight prior $\eta \sim \mathcal{N}(0.0715, 0.010^2)$ would give the same answer.)

**Goals target:** `gf_xg` / `ga_xg` — periods 1–4, **empty-net goals excluded** (5.9% of goals). Empty-net goals are scored by the team already leading, so they never change the winner; they reflect game state rather than team strength; and excluding them matches the xG model. Track B uses the same target.

**Expect wide posteriors.** Two teams meet three or four times a season, so even five seasons give only about 15 games per pair. Track A is a correctness check, not a source of useful team ratings.

**Sampler:** Adaptive Metropolis, three phases:

1. **Burn-in** ($n_b = 5000$): isotropic proposal $\Sigma_p = \sigma_0^2 I$ with $\sigma_0 = 0.1$
2. **Adapt:** estimate posterior covariance from burn-in, set $\Sigma_p = s \cdot \hat{\Sigma}$ with $s = (2.38)^2/d$
3. **Sample:** main run with fixed adapted covariance

**Reuses:** your soil sampler module, standardization machinery, diagnostics (trace plots, ACF, acceptance rate, $\hat{R}$).

**Purpose:** proof of life, reference posteriors, bug-catching for Track B.

### Track B — Hierarchical PyMC Model

**Scope:** full league simultaneously, on a recent window (v1: `FIT_SEASONS` = 2024-25 and 2025-26, static team ratings), fit to **regulation goals** (periods 1–3, no empty-netters).

**Model:**

$$
\begin{aligned}
\mu_{\text{off},t} &\sim \mathcal{N}(0, \sigma_{\text{off}}^2) \quad \text{for each team } t \\
\mu_{\text{def},t} &\sim \mathcal{N}(0, \sigma_{\text{def}}^2) \quad \text{for each team } t \\
\alpha_0 &\sim \mathcal{N}(\log 3,\ 0.5^2) \quad \text{league baseline scoring rate} \\
\alpha_{\text{home}} &\sim \mathcal{N}(0, 1) \\
\log \lambda_{\text{home}} &= \alpha_0 + \alpha_{\text{home}} + \mu_{\text{off},h} - \mu_{\text{def},a} - \gamma_a \\
\log \lambda_{\text{away}} &= \alpha_0 + \mu_{\text{off},a} - \mu_{\text{def},h} - \gamma_h \\
\rho &= e^{\tau} - 1, \quad \tau \sim \mathcal{N}(0,\ 0.5^2) \quad \text{tie inflation}
\end{aligned}
$$

$$
P(G_{\text{home}} = h, G_{\text{away}} = a) = \frac{\text{Pois}(h \mid \lambda_{\text{home}})\,\text{Pois}(a \mid \lambda_{\text{away}})\,(1 + \rho\,[h = a])}{1 + \rho\,T}, \quad T = \sum_k \text{Pois}(k \mid \lambda_{\text{home}})\,\text{Pois}(k \mid \lambda_{\text{away}})
$$

**Tie inflation.** Two independent Poisson scores end regulation tied about 17% of the time; real NHL games are tied after 60 minutes 23.7% of the time (2024-26). Late in tied games both teams play safe for the guaranteed point, and trailing teams press to equalize, so scores are not independent. $\rho$ multiplies every tied score by $1 + \rho$ and renormalizes ($T$ is the tie probability under plain Poisson). $\rho = 0$ is independent Poisson. In PyMC the two Poisson terms stay as observed variables and the correction $\log(1 + \rho[h = a]) - \log(1 + \rho T)$ is added with `pm.Potential` ($T$ summed over $k = 0..15$). `sample_scores` draws from this model exactly (a mixture: with probability $\rho T/(1 + \rho T)$ force a tie with $k \propto \text{Pois}(k \mid \lambda_h)\text{Pois}(k \mid \lambda_a)$, otherwise independent Poisson). Phase 6's exact win-probability model uses the same tie-inflated table; its score outputs come from the score-state simulator instead (see Phase 6). Why it matters: it sets how often games reach OT, which moves regulation-win probabilities and, above all, the number of loser points in simulated standings.

**Hyperpriors:** $\sigma_{\text{off}}, \sigma_{\text{def}} \sim \text{HalfNormal}(0.3)$ (team effects on the log scale are about ±0.1, so 0.3 is loose but rules out silly values).

**Goalie term is a fixed input, not estimated.** $\gamma_a$ and $\gamma_h$ are known numbers per game: $\gamma = -\log \theta$, where $\theta$ is the Phase 3 goalie rating (`goalie_rating_as_of`, posterior mean) of the goalie who **started** that game, computed as of that game's date so no game uses later information. Positive $\gamma$ = better goalie = fewer goals against. A goalie with no rating history gets $\gamma = 0$ (league average). This works like $\alpha_0$ and $\eta$ in Track A: a precisely known term held fixed.

Why not estimate a goalie effect inside Track B: from goal counts alone, a goalie who plays most games for one team is always seen alongside that team's defense, so the model can't tell good defense from good goaltending (only backups and trades separate them). It would also rate goalies a second time, in a place that can't see shot quality. With $\gamma$ fixed, $\mu_{\text{def}}$ means team defense *apart from* goaltending, and each goalie is rated once, by the model that sees shot quality.

**Cost:** Track B treats the goalie ratings as exact and ignores their uncertainty. The Phase 3 posterior sd is about ±0.03 on the log scale, small next to team effects, so this is acceptable for v1. The v2.1 hierarchical xG layer plans a fully Bayesian goalie effect whose posterior draws can replace the fixed values and carry that uncertainty through.

**Why the global intercept $\alpha_0$ is required.** Every team effect has a zero-mean prior (and the fixed goalie offsets average about 0), so without $\alpha_0$ the model is centered at $\log \lambda = 0$, i.e. about 1 goal per team per game instead of about 3. The team effects would have to shift together to make up the difference, fighting their own priors. $\alpha_0$ carries the league scoring level; the team effects and goalie offsets are then deviations from it. Its prior is centered on $\log 3 \approx 1.1$ and is wide enough to let the data decide.

**Remaining soft identifiability.** $\alpha_0$ can still trade off against the average of the $\mu_{\text{off}}$'s and the $\mu_{\text{def}}$'s. (The fixed $\gamma$'s average about 0, since the Phase 3 ratings are season-adjusted to league GA ÷ xGA = 1.) The zero-mean priors resolve this softly. On the real fit the correlation is strong ($r = -0.81$) but harmless: $\alpha_0$ still has ESS 2,239. Switch to sum-to-zero team effects (`pm.ZeroSumNormal`) only if $\alpha_0$'s ESS ever drops.

**Sampler:** NUTS. Write every random effect in non-centered form from the start ($\mu_{\text{off},t} = \sigma_{\text{off}}\, z_t$, $z_t \sim \mathcal{N}(0,1)$), because centered hierarchical models with small $\sigma$ produce funnel geometry and divergences.

**Output:** ArviZ NetCDF with the full joint posterior across all teams' offense and defense, plus $\alpha_0$, $\alpha_{\text{home}}$, the $\sigma$'s and $\rho$. The team indices and fixed $\gamma$ values used are stored in `constant_data`, the goals in `observed_data`.

**Purpose:** the actual production model.

**v1.1 (Phase 7, adopted).** The model above is v1 (goals only). Phase 7 replaced it with v1.1, which learns the team ratings mainly from regulation xG (Gamma likelihood) and fits goals on top with small finishing effects, on a 1-year window. Same outputs under the same names; see Phase 7 for the model and results.

### Cross-Validation Between Tracks

The two tracks only *should* agree when they are fit to the same data with the same likelihood. Comparing Track A on head-to-head games with full Track B on the whole league compares different datasets and different priors, so a gap between them proves nothing. The check is therefore done in three steps, from strict to loose.

**1. Synthetic recovery (both tracks).** Simulate a full league season from Track B's generative model with known parameters.

- Fit Track B and check that the posteriors cover the true team effects (simulated with the same fixed goalie offsets).
- Fit Track A on 2–3 team pairs from the same simulated games and check that its $d_A, d_B$ intervals cover the true differences.

**2. Like-for-like on real data (the strict agreement test).** For 2–3 team pairs, fit Track B in a *matched* configuration on the same $(A, B)$ head-to-head games Track A sees:

- no pooling ($\sigma_{\text{off}}, \sigma_{\text{def}}$ fixed at 1),
- goalie term and tie inflation switched off (Track A has neither),
- $\alpha_0$ and the home term fixed to the same values as Track A,
- the same goals Track A used (periods 1–4).

The priors differ slightly (Track A: uniform on $[-1, 1]$ for each $d$; matched Track B: $\mathcal{N}(0,1)$ on each team term, so $\mathcal{N}(0,2)$ on each $d$), which moves the means by about 0.001 with 18 games. A truly flat prior would leave NUTS wandering along the two directions the data can't pin down.

The two posteriors on $d_A, d_B$ should then match to within Monte Carlo error. A real disagreement here means a bug in the likelihood, the parameterization or one of the samplers.

**3. Pooled Track B vs Track A (qualitative only).** Fit full Track B on league data and compute its posterior on the same $d_A, d_B$. It should be much narrower and shrunk toward zero, and it should usually sit inside Track A's wide interval. A pooled interval far outside Track A's is a flag to investigate, not an automatic failure, since pooling and the extra league data are supposed to move it.

---

## Build Sequence

Realistic for a side project with focused sessions a few times a week. Roughly two months to a usable single-game predictor.

### Phase 1 — Data Foundation (Weeks 1–2) — DONE

- ~~Expand `scripts/load_shots.py` to iterate over the full league schedule.~~ Done as `scripts/load_all.py`: all 32 teams, 5 seasons.
- ~~Verify shot data integrity.~~ `notebooks/data_sanity.ipynb`, sections 1–13, all passing.
- ~~Backfill 2–3 seasons.~~ 2021-22 through 2025-26 (6,560 games).

**Added beyond the original plan:**

- `game_events` table (full play-by-play, every event type), needed for rush detection.
- `shot_context` table with rebound / rush flags, prior event type and time since prior event.
- Fixes found by the checks: goal-shooter repair, blocked-shot zone frame, no-stoppage rule (see Data Contract → Known Data Quirks).
- External check against Natural Stat Trick (`scripts/nst_spotcheck.py`, `scripts/rush_event_audit.py`).

## Phase 2 — xG Model (Week 3) — DONE

**Files:**

- `src/processing/xg_processing.py` — `build_xg_features(conn)`: one row per shot, 13 features
- `src/models/deterministic/xg.py` — `ExpectedGoalsModel` with `.fit()`, `.predict()`, `.evaluate()`, `.calibration()`, `.feature_importance()`, `.save()`, `.load()`
- `src/pipelines/train_xg.py` — run with `py -m src.pipelines.train_xg [--final]`
- `notebooks/xg_features_check.ipynb`, `notebooks/xg_model_dev.ipynb`

**Features (13):** distance, angle, shot type (grouped), rebound flag, rush flag, previous event type, time since previous event, skater difference, total skaters, own goalie pulled, distance from previous event, speed from previous event, rebound angle change.

**Model:** LightGBM, log-loss objective. `num_leaves=31`, `min_child_samples=200`, `learning_rate=0.03`, `subsample=0.8`, `colsample_bytree=0.8`. A 9-combination grid search found no better setting (top six within 0.00005 log loss; 63 leaves worse).

**Season roles:** train 2021-22 to 2023-24, validate 2024-25 (early stopping picks 413 trees), then refit on 2021-22 to 2024-25 and test once on 2025-26. Recency weights 1.0 / 1.25 / 1.5 / 2.0, oldest to newest. Random seed 42.

**Results on the 2025-26 test season:**

| | **Baseline (distance + angle, logistic)** | **Final LightGBM** |
|---|---|---|
| AUC | 0.714 | **0.769** |
| Log loss | 0.2307 | **0.2167** |
| Predicted vs actual goals | +1.4% | +2.5% (7,770 vs 7,578) |
| Calibration | Mid-range under, top over | Within 0.8 points in every decile |

What moved the score, in order: LightGBM over logistic regression; previous-event features; recency weighting; shot-type grouping (fixed the goal total).

**MoneyPuck comparison (2025-26, 5-on-5, team-season totals, 32 teams):** xGF r = 0.970, xGA r = 0.940, total xG ratio 1.000. MoneyPuck blocks scripted downloads; save `teams.csv` by hand to `data/external/`.

**Saved models (`output/models/`), version v1:**

- `xg_lgbm_v1_20212022_20242025.pkl` + `_metrics.json` — the evaluated model
- `xg_lgbm_v1_live.pkl` + `_metrics.json` — same settings, trained on all five seasons, for scoring new games
- `XG_MODEL_PATH` in `config/settings.py` names the active model; load it with `load_active_model()`.
- Metrics files record version, seed, settings, features, training seasons, data cutoff and library versions.
- `train_xg.py` refuses to overwrite an existing file; bump `MODEL_VERSION` in `xg.py` when features or settings change.

**Gaps against the original plan:**

- MoneyPuck validation was done on team-season totals, not per-game totals. Per-game is stricter and still to do.
- Method is named `.predict()`, not `.predict_proba()`.

## Phase 3 — Team Strength & Goalie Aggregates (Week 4) — DONE

**Files:**

- `src/pipelines/score_xg.py` — scores every shot into the `shot_xg` table. Run with `py -m src.pipelines.score_xg [--season YYYYYYYY]`.
- `src/processing/team_strength.py` — `team_game_xg(conn)`, `rolling_strength(tg, window=25)`, `strength_as_of(tg, date)`
- `src/processing/goalie.py` — `goalie_game_xg(conn)`
- `src/models/stochastic/goalie.py` — `goalie_rating_as_of(gg, date, k=800, lookback_days=730, season_adjust=True)`
- `notebooks/phase3_check.ipynb` — correctness checks plus the tests that chose the window and `k`

**Scoring every shot (`shot_xg`).** Each season 2021-22 to 2025-26 is scored by a leave-one-season-out model (trained on the other four seasons, 413 trees, same settings as v1), so no shot is scored by a model that saw it. Seasons the live model never trained on (2026-27 onward) are scored by the live model. xG / goals by season: 1.032, 1.031, 0.974, 0.969, 1.024 (see *Season-level xG offset* in Open Questions).

**Team strength.** One row per team per game (`team_game_xg`): xGF, xGA, goals for and against (periods 1–4, empty-net goals included) and game length in minutes (overtime counted). `rolling_strength` adds each team's estimate going *into* every game from its previous 25 games: xGF/60 and xGA/60 as total xG ÷ total minutes × 60, and their variance (game-to-game variance ÷ games). Windows span seasons. `strength_as_of` gives the same estimate for every team on any date.

**Goalie rating.** `goalie_game_xg` credits each shot to the goalie in net, so a pulled starter and his replacement each carry only the shots they faced. The rating is Gamma-Poisson on goals allowed per expected goal:

$$GA \sim \text{Poisson}(\theta \cdot xGA), \qquad \theta \sim \text{Gamma}(k, k), \qquad \theta \mid \text{data} \sim \text{Gamma}(k + GA,\ k + xGA)$$

$\theta < 1$ is better than average. Each season's xGA is first rescaled so league GA ÷ xGA = 1.0 within that season (using only games before the rating date). Output per goalie: posterior mean and variance of $\theta$, a 90% credible interval, raw GSAx and games. This is the swappable goalie interface; on Track B's additive log scale, $\gamma_g \approx -\log \theta_g$ (positive = better).

**Results (`phase3_check.ipynb`):**

Correctness: two rows per game (13,120), league xGF = league xGA, xGF and goalie xGA per season match `shot_xg` exactly, every goalie-game named, and a hand-computed pre-game estimate matches `rolling_strength` (no leakage).

Team strength predicting the next game (6,381 games where both teams had 10+ games of history):

| Window | Correlation with goal diff (xG) | (goals) | Picks winner (xG) |
|---|---|---|---|
| 10 | 0.233 | 0.168 | 58.2% |
| 25 | **0.251** | 0.206 | **59.3%** |
| 50 | **0.251** | 0.240 | 59.1% |

xG-based estimates beat goals-based ones at every window. 25 and 50 tie; 25 is kept because it reacts faster.

Goalie rating predicting the next season (2022-23 to 2025-26 pooled, 253 goalie-seasons with 20+ games, correlation margin of error about ±0.12):

| k | Weighted error (adjusted) | Correlation (adjusted) | Correlation (unadjusted) |
|---|---|---|---|
| 1 (raw record) | 0.0284 | 0.037 | 0.043 |
| 100 | 0.0138 | 0.100 | 0.094 |
| 400 | 0.0120 | 0.120 | 0.105 |
| **800** | **0.0119** | 0.124 | 0.106 |
| 1600 | 0.0119 | 0.125 | 0.106 |

The season adjustment improves error and correlation from k = 100 up. Error is lowest at k = 800 and flat from about 400 to 1,600. The signal is real but weak: a goalie's past two seasons are a modest guide to his next one, consistent with public hockey analytics. With k = 800, ratings for big-sample goalies span roughly 0.96 to 1.04.

**Settings kept:** team window 25 games; goalie `k = 800`, `season_adjust=True`, `lookback_days = 730`.

**Notes:**

- `games` in goalie output counts appearances, so a short relief stint counts as one game. The rating itself is weighted by xGA faced, not games.
- Utah (`UTA`) has a different abbreviation from Arizona, so its team-strength history starts fresh in 2024-25.
- Goalie names come from `goalie_game_stats` (boxscores), which covers every goalie who played, not only current rosters.

## Phase 4 — Track A Toy MCMC (Week 5) — DONE

**Files:**

- `src/models/stochastic/am_sampler.py` — `adaptive_metropolis`, `autocorrelation`, `plot_traces`, `plot_acf`, ported unchanged from the soil project
- `src/models/stochastic/toy_mcmc.py` — `league_baseline(tg)`, `pair_games(tg, team_a, team_b)`, `make_log_posterior(games, alpha0, eta)`, `simulate_pair_games(n_games, d_A, d_B, alpha0, eta, seed)`
- `src/pipelines/fit_toy.py` — fits one pair, 4 chains. Run with `py -m src.pipelines.fit_toy COL DAL [--overwrite]`.
- `notebooks/phase4_toy.ipynb` — function checks and the four recovery tests

**Model.** Two parameters, uniform prior on $[-1, 1]^2$, Poisson likelihood on `gf_xg` / `ga_xg`:

$$\log \lambda_A = \alpha_0 + \eta \cdot \text{home}_A + d_A, \qquad \log \lambda_B = \alpha_0 + \eta \cdot \text{home}_B + d_B$$

with $\alpha_0 = 1.0291$ (2.799 away goals/game) and $\eta = 0.0715$ (home teams score 7.4% more, 3.006 goals/game) fixed from league data by `league_baseline`. Empty-net goals are 5.9% of all goals.

**Exact posterior (used as a check).** With a flat prior on $d$, the posterior of $e^d$ is $\text{Gamma}(Y, B)$, where $Y$ = total goals and $B = \sum_g e^{\alpha_0 + \eta \cdot \text{home}_g}$. So $E[d] = \psi(Y) - \log B$ and $\text{sd}(d) = \sqrt{\psi_1(Y)}$. The sampler is compared against this, not just against itself.

**Results (`phase4_toy.ipynb`, truth $d_A = 0.15$, $d_B = -0.10$):**

| Test | Result |
|---|---|
| `log_post` vs scipy Poisson (differences) | Match to 6 decimals |
| 1. One fit, 15 games, 4 chains | Truth inside both 90% intervals; acceptance 0.35–0.37; R-hat 1.0001; ACF below 0.05 by lag 11 |
| 2. 500 games | Posterior narrows 5.7× and 6.1× (expected about 5.8×) |
| 3. Coverage, 100 seeds × 15 games | 90/100 and 96/100 (target 90, normal range 84–96) |
| 4. MCMC vs exact | Means within 0.002, sds within 1% |

In Test 1 the truth sits about 0.9 sd from the posterior mean of $d_B$. That comes from the 15 simulated games (B scored below its true rate by chance), not from the sampler: MCMC matches the exact posterior for that data. Coverage is what tests calibration.

**Real pair, COL vs DAL** (18 games, 2021-11-26 to 2026-04-04, COL home in 9; `fit_toy.py`, 4 × 50,000 draws):

| | Mean | 90% interval | Rate vs league average | Exact mean |
|---|---|---|---|---|
| $d_A$ (COL scoring vs DAL) | +0.114 | [−0.106, +0.325] | ×1.12 | +0.113 |
| $d_B$ (DAL scoring vs COL) | −0.055 | [−0.296, +0.173] | ×0.95 | −0.054 |

R-hat 1.0001–1.0002, ESS about 26,000–28,000, acceptance 0.34–0.36. Both intervals cover 0: 18 games can't separate the two teams, as expected.

**What Track A is for downstream.** Nothing in Phases 6–8 reads Track A's numbers; they use Track B. Track A's value is what it proves and what it provides as a reference:

- **Phase 5 (Track B) is checked against it.** The matched Track B fit on the same head-to-head games, with the same fixed $\alpha_0$ and $\eta$, must reproduce the `toy_<A>_<B>.nc` posteriors. A gap means a bug in Track B's likelihood or parameterization, found before Track B is trusted league-wide.
- **The simulator and its exact answer are reusable.** `simulate_pair_games` and the Gamma check extend to Track B's synthetic-recovery test (Cross-Validation step 1).
- **It fixed the shared conventions** every later model uses: the goals target (`gf_xg`), the fixed-league-term approach, the posterior file format (NetCDF with observed data and settings attached) and the diagnostics (R-hat, ESS, acceptance).

- **The sampler is verified,** so if it is ever needed again (for example, PCE-based inference if a slow possession-level simulator replaces the Poisson game model) it is ready.

**Settings kept:** 4 chains, 5,000 burn-in + 50,000 draws, $\sigma_0 = 0.1$, seed 42 (+ chain number), starts spread over the prior box.

**Notes:**

- **ArviZ 1.x** (current; installed with Python 3.14) returns an xarray `DataTree` instead of `InferenceData`, and `az.from_dict` takes one nested dictionary. `fit_toy.py` handles both 0.x and 1.x. Version 1.x drops `attrs` passed to `from_dict`, so model settings are written to `posterior.attrs` directly.
- **Writing `.nc` files needs `h5netcdf` and `h5py`:** `py -m pip install arviz h5netcdf h5py`.
- Summary statistics in `fit_toy.py` (mean, sd, equal-tailed 90% interval) are computed with numpy so they match the notebook; R-hat and ESS come from ArviZ.
- Acceptance of about 0.35 is right for two parameters (0.234 is the many-dimension optimum).

## Phase 5 — Track B Hierarchical PyMC (Week 6) — DONE

**Files:**

- `src/processing/game_table.py` — `starting_goalies(conn)`, `regulation_goals(conn)`, `goalie_gamma(gg, dates)`, `build_game_table(conn, seasons)` → `(gt, teams)`: one row per game with team indices, goals (periods 1–4 and regulation), starters and their fixed $\gamma$
- `src/models/stochastic/hierarchical.py` — `build_model(gt, teams, matched=False, ..., goal_cols=GOAL_COLS, tie_inflation=True)`, `sample(model, seed)`, `sample_scores(lam_home, lam_away, rho, rng)`, `simulate_league(...)`, `team_ratings(idata, teams)`
- `src/pipelines/fit_hierarchical.py` — `py -m src.pipelines.fit_hierarchical [--seasons ...] [--overwrite]`; writes `hier_<first>_<last>.nc`, `_meta.json`, `_ratings.csv`, `_trace.png` to `output/posteriors/`
- `notebooks/phase5_check.ipynb` — eleven sections of checks (below); section 11 tests the score-effects explanation
- `scripts/test_pymc.py` — install check: tiny Poisson fit against the exact answer

**Install:** `py -m pip install pymc` works on Python 3.14 (PyMC 6.3.2, PyTensor 3.3.4). PyTensor found the MSYS2 g++ compiler (`C:\msys64\ucrt64\bin`), so sampling runs compiled. Sampling uses `cores=1` (chains one after another) on Windows.

**Game table** (2024-25 + 2025-26): 2,624 games, 32 teams, 164 games each. Starter found for every side. Regulation goals per game: home 2.867, away 2.679. **23.7% of games tied after regulation**; 15.3% decided by an OT goal (the rest by shootout). Goalie $\gamma$ from −0.050 to +0.066; 0.3% of sides have no rating history ($\gamma = 0$). Recomputing one starter's $\gamma$ from earlier games only matches exactly (no leakage).

**Results (`phase5_check.ipynb`):**

| Check | Result |
|---|---|
| Synthetic recovery (3 fake leagues × 2,480 games, true $\rho = 0.5$) | Team effects: 88.5% of 192 90% intervals cover the truth; league terms 15/15; $\rho$ estimated 0.61, 0.44, 0.54; 0 divergences |
| Matched check vs Track A (COL–DAL, 18 games) | $d_A$ +0.1113 vs +0.1141, $d_B$ −0.0525 vs −0.0553; sds within 1% |
| Sampler (real fit, about 50 s) | R-hat ≤ 1.0037, ESS ≥ 1,570, 0 divergences, BFMI 0.62–0.71 |
| Home ice | $\alpha_{\text{home}}$ = 0.0713 [0.043, 0.099]; direct league ratio on the same games 0.0676 |
| Scoring level | $\alpha_0$ = 0.992 [0.958, 1.025] (regulation away goals ≈ 2.70) |
| Team spread | $\sigma_{\text{off}}$ = 0.068, $\sigma_{\text{def}}$ = 0.064: a typical team is about ±7% on offense and on defense |
| Tie inflation | $\rho$ = 0.51 [0.40, 0.63]: regulation ties about 51% more likely than independent scoring |
| Sanity | Net rating vs goal differential r ≈ 0.998; vs **xG differential r ≈ 0.79** (never seen by the model) |

Top teams by net rating: COL (about +0.18), TBL, DAL, CAR, VGK; bottom: SJS (about −0.17), CHI, VAN, NSH, ANA. 90% intervals are about ±0.09 for every team because every team gives the model the same data (164 games, about 450 goals each way); the middle of the league overlaps heavily. Extreme goal differentials are shrunk (SJS −0.77 goals/game → −0.17; COL +0.77 → +0.18).

**Posterior predictive check** (every game re-simulated from 500 posterior draws; regulation goals):

| Statistic | Actual | Model 90% | Plain Poisson 90% |
|---|---|---|---|
| Goals per team-game | 2.773 | [2.724, 2.822] ✓ | [2.744, 2.843] ✓ |
| Variance ÷ mean | 0.922 | [0.961, 1.031] ✗ | [0.995, 1.066] ✗ |
| Share 0 goals | 0.052 | [0.054, 0.066] ✗ | [0.058, 0.070] ✗ |
| Share 5+ goals | 0.138 | [0.137, 0.155] ✓ | [0.144, 0.164] ✗ |
| Home win in regulation | 0.420 | [0.393, 0.433] ✓ | [0.426, 0.468] ✗ |
| Tied after regulation | 0.237 | [0.219, 0.256] ✓ | [0.159, 0.182] ✗ |

Tie inflation fixes ties and regulation home wins. What remains: each team's regulation goals are about 8% less spread than Poisson (more 2–3 goal games, fewer shutouts), and one-goal margins are more common than the model gives (home +1: 18.3% actual vs 14.7%).

**Why (section 11, from play-by-play):**

| Test | Result |
|---|---|
| Variance ÷ mean by periods played (no empty-netters) | Period 1: 0.981 (inside Poisson range 0.975–1.037) → periods 1–2: 0.960 → regulation: 0.922. The squeeze builds up during the game. |
| Scoring vs each team's own Track B pace, by score state | Trailing 2+: 1.11 · trailing 1: 1.07 · tied: 0.96 · leading 1: 0.94 · leading 2+: 0.93 (each ±0.02–0.04) |
| Regulation variance ÷ mean with empty-net goals put back | **1.010, inside the Poisson range** (0.986–1.053); removing empty-netters causes about 90% of the gap |

So **score effects are real** (trailing teams score about 7–11% above their normal pace, leaders about 6–7% below), but **most of the "too bunched" result comes from the goal definition**: empty-net goals go to leading teams, so leaving them out trims the high end of the counts. With them included, regulation goals are Poisson-like.

**Consequences:** no change to Track B (ratings measure normal-play strength, and empty-net goals never change the winner).

**Correction (found in Phase 6).** Reading this as "mostly a counting artifact" was wrong. Real regulation goals without empty-netters are compressed (0.92) by score effects, and empty-net goals then add spread back (1.01); the two are the same late-game behavior. Track B's regulation goals are Poisson-like, so adding empty-net goals on top over-spread final scores (1.06–1.07 vs 0.98). Phase 6 therefore built the score-state simulator for score outputs (see Phase 6).

**How the model got here** (each change found by a check):

1. First fit on periods 1–4 goals: the predictive check showed 8.5% "ties" (only shootout games) vs 16–18% simulated, because OT goals were counted as ordinary scoring. → Fit regulation goals; Phase 6 simulates OT.
2. Regulation fit: ties 23.7% actual vs 16–18% simulated, home regulation wins too high. → Tie inflation $\rho$.
3. Dispersion still below Poisson. → Section 11 traced it to score effects plus the empty-net exclusion. → No change to Track B; Phase 6 score-state simulator for score outputs.

**Settings kept:** window 2024-25 + 2025-26; priors as in *Track B*; NUTS 4 chains × (1,000 tune + 1,000 draws), `target_accept = 0.9`, seed 42.

**Notes:**

- **Windows file lock.** `az.from_netcdf` keeps the file open; a notebook kernel that loaded a posterior locks it and `--overwrite` fails (`GetLastError() = 33`). The notebook's `load_posterior()` reads into memory and closes the file. Restart

any old kernel (or close VS Code) to release a lock. A safer save (write to `.tmp`, then swap) is written but not yet installed in `fit_hierarchical.py`.
- `fit_hierarchical.py` sets matplotlib's file-only backend inside `main()`, not at import, so notebooks that import from it keep inline plots.
- An OT goal after an *untied* regulation can happen when a removed empty-net goal hid a real regulation tie (leader scores into the empty net, trailer scores twice). The check allows up to 1% of games.
- Sampling took about 13 s with plain Poisson and about 50 s with tie inflation (the tie sum runs over 16 scores per game).

## Phase 6 — Game Model + Monte Carlo (Week 7) — DONE

**Files:**

- `src/processing/late_game.py` — `game_results`, `mean_rates`, `fit_ot_factor`, `shootout_home`, `empty_net_table`, `late_game_params`. `py -m src.processing.late_game` → `output/models/late_game.json`
- `src/processing/score_state.py` — `load_events`, `measure`, `summarize`, `score_state_params`. `py -m src.processing.score_state` → `output/models/score_state.json` (includes the simulator's calibrated `scale`)
- `src/models/game_model.py` — `load_track_b`, `default_starter`, `goalie_offset`, `matchup_rates`, `regulation_table`, `outcome_probs`, `final_score_table`, `predict_matchup`
- `src/models/stochastic/montecarlo.py` — tie-inflated simulator (`simulate_games`, `simulate_matchup`, `summarize_sims`) and score-state simulator (`simulate_regulation_score_state`, `simulate_games_score_state`, `simulate_matchup_score_state`, `calibrate_scale`)
- `src/analysis/aggregation.py` — `fair_american_odds`, `outcome_table`, `likely_scores`, `totals_and_lines`, `final_from_sims`, `format_prediction`, `result_to_json`
- `src/pipelines/predict_game.py` — `py -m src.pipelines.predict_game HOME AWAY [--date YYYY-MM-DD] [--home-goalie ID] [--away-goalie ID] [--score-sims N] [--check-sims N] [--save]`; saved predictions go to `output/predictions/<date>_<away>_at_<home>.json`
- `notebooks/phase6_check.ipynb` — nine sections (below)

**Rebuild order** whenever the data or the fit changes (each step uses the previous one's output; since Phase 7 the fit is v1.1 on the last 365 days and becomes the active fit): 0. After loading new games: `py .\scripts\load_all.py`, then `py -m src.pipelines.score_xg --season <season>` (the fit stops if this was skipped)

1. `py -m src.pipelines.fit_hierarchical` (add `--overwrite` to refit the same end date)
2. `py -m src.processing.late_game`
3. `py -m src.processing.score_state`

**In-season routine (2026-27):**

- **Every morning:** `load_all.py` → `score_xg --season 20262027` → `make_cards postgame --pending` (cards for last night's games) → `predict_day` (forward-test log + a pregame card per game; add `--goalie TEAM=NAME --replace` (last name or id) when starters are confirmed, which redraws that card).
- **Weekly:** `load_all.py` → `score_xg --season 20262027` → `fit_hierarchical` → `late_game` → `score_state` → `build_pce` (optional) → `score_forward`.

**Two models, each for what it does best:**

| Output | Source | Why |
|---|---|---|
| P(win), regulation / OT / shootout split, with 90% intervals | **Exact model** (`game_model.py`): tie-inflated regulation table (16 × 16) per posterior draw, then OT and shootout formulas | Analytic and fast; one P(win) per draw, so the mean is the prediction and the spread is the uncertainty about team strength (game randomness is already inside each P) |
| Likely final scores, totals, puck lines, margins | **Score-state simulator** (100,000 games per prediction) | Reproduces real score shapes (section 9) |

The prediction header prints the simulator's win share next to the exact one as a running consistency check.

**Game flow:**

1. **Regulation.** Rates from Track B: $\lambda = \exp(\alpha_0 [+\alpha_{\text{home}}] + \mu_{\text{off}} - \mu_{\text{def,opp}} - \gamma_{\text{opp goalie}})$, tonight's starters' $\gamma$ from the Phase 3 rating as of game day (default starter: most starts in the team's last 10 games).
2. **Overtime** if tied: $P(\text{OT goal}) = 1 - e^{-k(\lambda_h+\lambda_a)\cdot 5/60}$, home scores it with probability $\lambda_h/(\lambda_h+\lambda_a)$.
3. **Shootout** if still tied: 50/50.
4. **Official final score:** empty-net goals plus 1 for the OT or shootout winner.

**Late-game parameters (`late_game.py`, 2024-26):**

| Quantity | Value |
|---|---|
| How games ended | Regulation 77.2%, OT 15.3%, shootout 7.5%; home won 54.2% |
| OT factor $k$ | **2.39** [2.20, 2.61]: 3-on-3 scores at about 2.4× the regulation rate. Fitted share of OT games decided in OT 67.1% vs actual 67.2% |
| Who scores in OT | Home 52.1% vs 51.9% expected from ratings (no separate OT home edge needed) |
| Shootout | 196 games, home 49.5% [43.6, 55.3], p = 0.94 vs 50/50 → 50/50 |
| Leader's empty-net goals by regulation margin (0 / 1 / 2+) | Margin 1: 51.0 / 40.9 / 8.1% · margin 2: 29.5 / 68.7 / 1.8% · margin 3+: 79.1 / 20.9 / 0% (used by the tie-inflated simulator) |

**Score-state parameters (`score_state.py`, from `situation_code` goalie digits):**

| Situation | Value |
|---|---|
| Normal play (both goalies in), goals vs own Track B pace | Trailing 2+: 1.057 · trailing 1: 1.036 · tied: 0.974 · **tied, last 5 min of P3: 0.768** · leading 1: 0.958 · leading 2+: 0.956 |
| Goalie pulled, trailer's 6-on-5 factor | 2.72 (down 1) · 2.74 (down 2) · 2.63 (down 3+) |
| Goalie pulled, leader's empty-net goals | 0.358 / 0.388 / 0.319 per minute (down 1 / 2 / 3+) |
| Share pulled by 2:00 / 1:00 left (no goals in between) | Down 1: 44% / 98% · down 2: 92% / 98% · down 3+: 29% / 31% |
| Empty-net goals outside a late trailing pull | 113 (about 11%; delayed penalties, early pulls; not simulated) |

**Score-state simulator.** Regulation in 360 slices of 10 seconds. Each slice: (1) a trailing team in the last 6 minutes may pull its goalie (measured hazard by margin and 30-second bin); (2) rates: pulled trailer = own rate × 6-on-5 factor, leader facing an empty net = empty-net rate, otherwise own rate × score-state factor; (3) goals; after any goal both goalies return. **No tie inflation**: extra ties come from "tied late" caution and 6-on-5 equalizers. One overall `scale` (calibrated on the fit games) keeps average goals with a goalie in net equal to Track B's rates. 10-second slices keep late pulls realistic; two goals in one slice happen about once in 15,000 slices.

**Results (`phase6_check.ipynb`):**

| Check | Result |
|---|---|
| Exact vs simulated (20 matchups × 50,000 games) | All outcomes agree, worst \|z\| < 4; probabilities sum to 1; final scores have no ties |
| History: home win | 54.2% actual vs 53.4% expected [51.8, 55.0] |
| History: regulation / OT / shootout | 77.3 / 15.3 / 7.5% actual vs 76.3 / 15.9 / 7.8% expected, all inside 90% ranges (regulation about 1 point low: the "hidden tie" effect in the exact model) |
| Calibration preview (in-sample) | Log loss 0.6758 vs 0.6896 for "home team 54%"; Brier 0.2414 vs 0.2482; picks the winner 57.3%; 7 of 10 deciles inside their range |

| Check | Result |
|---|---|
| Sanity | Home ice +3.3 points for two average teams; COL vs SJS 69.3% at home, 63.2% away; a ±0.04 goalie offset swings P(win) 3.7 points |
| Exact vs score-state P(win), 40 games | Mean absolute difference 0.5 points (simulation noise 0.35) |

**Score shape, all 2,624 games (section 9):**

| | Actual | Tie-inflated sim | Score-state sim |
|---|---|---|---|
| Past regulation | 22.75% | 23.50% | **22.79%** |
| Variance ÷ mean, regulation without empty-netters | 0.922 | 1.005 | **0.938** |
| Variance ÷ mean, regulation with empty-netters | 1.010 | 1.115 | **1.048** |
| Variance ÷ mean, final goals | 0.979 | 1.072 | **1.011** |
| One- / two- / three-goal games | 40.7 / 19.7 / 22.9% | 38.1 / 17.5 / 27.9% | **41.2 / 19.6 / 24.3%** |
| Empty-net share of regulation goals | 6.62% | 6.34% | **6.62%** |
| Mean final total | 6.167 | 6.155 | 6.198 |
| Home win | 54.2% | 53.0% | 52.9% |

The score-state simulator is closer on 9 of 11 rows, hits the past-regulation share exactly without a tie parameter (which also resolves the empty-net "hidden tie" mismatch for score outputs), and closes about 70% of the final-score spread gap. Part of the remaining 1.011 vs 0.979 is expected: each simulated game draws its ratings from the posterior, which adds a little spread.

**Example** (`predict_game COL DAL --date 2026-04-10`, Blackwood vs Oettinger by default): COL 55.8% (90% 50.0–61.6%), fair odds COL −126 / DAL +126; tied after regulation 23.8%.

**How Phase 6 got here:**

1. Exact model + tie-inflated simulator with an empty-net table: win and OT/SO shares matched history, but final scores were too spread (1.06 vs 0.98) with too many 3-goal margins.
2. That showed score effects matter for score outputs (correction to the Phase 5 reading). → Measured score effects, pull timing and 6-on-5 / empty-net rates; built the score-state simulator; switched score outputs to it.
3. A tie parameter on top of the score-state simulator was rejected: ties already match, and it would double-count them.

**Notes:**

- `predict_game.py` with today's date looks for the most recent starters; until 2026-27 games are loaded it uses the last starters on record. To predict this season's games: load 2026-27, then run the rebuild order above.
- The exact model's past-regulation share is 23.5% vs 22.8% real (empty-net "hidden ties"); harmless for P(win), and score outputs no longer use it.
- Calibration is too cautious at the extremes in-sample (predicted 43.2% → actual 37.3%; 59.6% → 69.1%; 63.4% → 68.8%). Unusual in-sample, where over-confidence is normal; leading suspect is static two-season ratings blurring teams that changed. First question for Phase 7. **Answered in Phase 7: it did not hold out of sample.** On held-out games the two-season v1 was *too confident* (slope 0.79).

## Phase 7 — Validation (Week 8) — DONE

**Files:**

- `src/validation/backtest.py` — `fit_window`, `predict_block`, `walk_forward(conn, season, window_days, refit_days=7, score_sims=5000, model="goals"|"xg", half_life=None, ...)`, `model_label` (e.g. `trackb_w730`, `trackbxg_w365`, `trackbxg_w365_hl90`)
- `src/validation/baselines.py` — `home_rate`, `elo` (tuned), `rolling_xg`, `all_baselines`

- `src/validation/scoring.py` — `log_loss`, `brier`, `accuracy`, `calibration_slope`, `calibration_table`, `score_models`, `paired_difference`, `by_group`, `total_goals_rps`, `score_outputs`
- `src/models/stochastic/hierarchical_xg.py` — Track B v1.1 (below): `build_model_xg(gt, teams, half_life_days=None, ref_date=None)`, `game_weights`, `simulate_league_xg`
- `src/processing/game_table.py` — added `regulation_xg(conn)`; the game table now carries `home_xg_reg` / `away_xg_reg`
- `src/pipelines/backtest_season.py` — `py -m src.pipelines.backtest_season --season 20252026 [--model goals|xg] [--window 365] [--half-life 90] [--refit-days 7] [--restart]` → `output/backtests/<label>_<season>.csv` (appended after every refit, so a stopped run resumes) and `<label>_<season>_scored.csv`
- `src/pipelines/run_baselines.py` — `py -m src.pipelines.run_baselines --seasons 20242025 20252026` → `baselines_<season>.csv` + `_info.json`
- `notebooks/phase7_check.ipynb` — finds every `trackb*_scored.csv` on its own; sections: data checks, model table per season and combined, paired head-to-head, calibration, by month/season, interval widths, score outputs, blends, market

**Backtest design (leak-free).** Test seasons 2024-25 and 2025-26 (1,312 games each). Before opening night: fit Track B on the `window_days` before it and measure the late-game and score-state parameters on those games only. Then refit every 7 days on the window before the refit date and predict the next week's games. Goalie offsets use the actual starters (known by game time, as they are for closing lines) rated as of the game date; each season's xG comes from a model that never trained on it. About 50 s per weekly refit (v1.1 about 2–3 min), about 26 refits per season.

**Competitors:**

| Model | How it predicts |
|---|---|
| Home rate | Home win share of earlier seasons, same for every game |
| Elo | Grid-tuned on earlier seasons only (k, home edge, off-season carry-over); first season is burn-in |
| Rolling xG | Logistic regression on the difference in each team's last-25-game xG rate (Phase 3), needs 10+ games |
| Track B v1 | Goals only, 1-year (`w365`) or 2-year (`w730`) window |
| Track B v1.1 | xG + goals, 1- or 2-year window, optional recency half-life |

**Track B v1.1 — ratings from xG and goals** (`hierarchical_xg.py`). Built in Phase 7 because v1 lost to rolling xG. Each game gives two observations per side (regulation, no empty-netters):

$$\log \nu_{\text{home}} = \beta_0 + \beta_{\text{home}} + \mu^{xG}_{\text{off},h} - \mu^{xG}_{\text{def},a}, \qquad xG_{\text{home}} \sim \text{Gamma}(\text{mean } \nu_{\text{home}},\ \text{shape } \kappa)$$
$$\log \lambda_{\text{home}} = \log \nu_{\text{home}} + \delta_0 + \delta_{\text{home}} + \phi_{\text{off},h} - \phi_{\text{def},a} - \gamma_a$$
$$(G_{\text{home}}, G_{\text{away}}) \sim \text{tie-inflated Poisson}(\lambda_{\text{home}}, \lambda_{\text{away}};\ \rho)$$

and the mirror image for the away side. $\mu^{xG}$ are shot-quality ratings, learned mostly from xG (the steadier signal). $\phi$ are finishing / team-save effects beyond xG and the goalie, tightly pooled ($\sigma_\phi \sim \text{HalfNormal}(0.05)$) so they stay near 0 unless the data clearly support them. The goalie $\gamma$ acts on goals only (goalies change conversion, not shot quality). Priors: $\beta_0 \sim \mathcal{N}(\log 2.7, 0.5^2)$, $\beta_{\text{home}} \sim \mathcal{N}(0, 0.5^2)$, $\delta \sim \mathcal{N}(0, 0.2^2)$, $\sigma \sim \text{HalfNormal}(0.3)$, $\log \kappa \sim \mathcal{N}(\log 9, 1)$, tie prior as v1. Both likelihoods are added with `pm.Potential`. The model also saves goal-scale ratings under the v1 names ($\alpha_0 = \beta_0 + \delta_0$, $\alpha_{\text{home}} = \beta_{\text{home}} + \delta_{\text{home}}$, $\mu = \mu^{xG} + \phi$), so the game model, simulators and backtest work unchanged. Optional recency weights $0.5^{\text{age/half-life}}$ multiply each game's log-likelihood.

Synthetic recovery (fake league, 2,480 games): correlation with the true goal-scale ratings 0.93 / 0.94 (offense / defense) vs 0.82 / 0.88 for goals-only v1 on the same data; 0 divergences. PyTensor sometimes prints "overflow encountered in dot" during tuning; those steps are rejected and harmless (no divergences).

**Results: P(home win), 2,624 held-out games** (log loss, lower is better; coin flip 0.6931):

| Model | 2024-25 | 2025-26 | Combined | Calibration slope [90%] |
|---|---|---|---|---|
| Home rate | 0.6870 | 0.6929 | 0.6900 | n/a |

| Model | 2024-25 | 2025-26 | Combined | Calibration slope [90%] |
|---|---|---|---|---|
| Elo (tuned) | 0.6721 | 0.6922 | 0.6822 | 0.64 (too confident) |
| **Rolling xG** | **0.6669** | **0.6838** | **0.6754** | 0.83 (too confident) |
| Track B v1, 2-yr | — | — | 0.6811 | 0.79 (too confident) |
| Track B v1, 1-yr | — | — | 0.6798 | 1.06 |
| **Track B v1.1, 1-yr** (adopted) | 0.6670 | 0.6877 | 0.6773 | **0.99** [0.80, 1.16] |
| Track B v1.1, 2-yr, half-life 180 d | 0.6673 | 0.6854 | 0.6763 | 1.20 [0.97, 1.41] |
| Track B v1.1, 1-yr, half-life 90 d | 0.6683 | 0.6841 | 0.6762 | 1.50 [1.24, 1.73] (too cautious) |
| Track B v1.1, 1-yr, half-life 60 d | 0.6700 | 0.6841 | 0.6771 | 1.61 [1.32, 1.89] (too cautious) |

Slope 1 = calibrated; below 1 = too confident; above 1 = too cautious.

**Paired comparisons** (same games, bootstrap 90% range; negative = first model better):

- Every v1.1 version clearly beats v1 and Elo.
- v1.1 vs rolling xG: 1-yr +0.0020 [−0.0020, +0.0058]; hl180 +0.0010; hl90 +0.0008 [−0.0031, +0.0046]; hl60 +0.0017. **All tied**: every range spans 0.
- hl90 vs hl60: −0.0009 [−0.0016, −0.0002]. The half-life versions vs the unweighted 1-yr: not significant.
- Averaging any v1.1 version with rolling xG beats that v1.1 alone by 0.0023–0.0031 (best blend 0.6738, with hl90). The two models make different mistakes.

**Score outputs** (over 5.5 / 6.5, puck lines, total-goals RPS from the score-state simulator): slightly better than predicting each market's historical rate. Totals are limited by season-to-season drift in league scoring, which a window fit can't see coming.

**What Phase 7 found:**

1. **The Phase 6 "too cautious" result did not hold out of sample.** On held-out games, v1 with two seasons was too confident (0.79): static ratings over two seasons treat old games as current. One season fixed calibration (1.06) and improved log loss.
2. **Goals alone are not enough.** v1 lost to rolling xG. Putting xG inside Track B (v1.1) closed the gap to a statistical tie, kept calibration (0.99) and beat v1 and Elo clearly.
3. **Recency helps only when teams change, and costs calibration.** Half-lives of 60–90 days matched rolling xG in 2025-26 (0.6841 vs 0.6838) but were worse in the steadier 2024-25 (0.6700 vs 0.6670), and made predictions too cautious: down-weighting games leaves fewer effective games, so every team's rating is less certain, and averaging over that pulls predictions toward 50%. A half-life forgets every team at the same rate whether it changed or not.
4. **No Track B version beats rolling xG,** and the blend still helps. What is still missing is recent form, handled properly: ratings that drift over time, at a rate the model estimates (v2 item 1).

**Adopted:** Track B v1.1, 1-year window, no half-life. Ties the best baseline, best calibrated model tested, no tuning knob. The rolling-xG blend is kept as a **benchmark, not a production feature**: it would give P(win) with no honest interval, ignore the starting goalie in half the average, and disagree with the simulator's P(win). If time-varying ratings work in v2, blending should stop helping; that is v2's test.

**How Phase 7 got here:**

1. Backtest v1 (1- and 2-year windows) against the baselines: v1 lost to rolling xG; 2-year was too confident. → One-year window; xG into Track B.
2. Built v1.1 (xG + goals): tied rolling xG, calibrated. Still behind in 2025-26. → Tested recency (half-lives 180, 90, 60 days).
3. Recency tied 2025-26 but hurt 2024-25 and calibration. → Kept v1.1 1-yr; time-varying ratings to v2.

**Not done in Phase 7:**

- **Market comparison.** Notebook section 10 is ready; it needs closing moneylines in `data/external/closing_lines.csv` (columns `date, home, away, home_ml, away_ml`). Remove the margin by dividing each implied probability by their sum

(about 1.04–1.05), then score like the models. A model that doesn't beat the closing line loses money once the margin is taken out, so this is information, not betting advice.
- Per-subset comparisons, the cost of a wrong default starter, the held-out home-win share and a forward test: all listed with the market comparison in *v2.0b Phase 7 follow-ups*.

**Switching production to v1.1.** `fit_hierarchical.py` now fits the same model and window as the backtest: default `--model xg --window 365`, games from `end − 365 days` up to `end` (default: the day after the last game in the database), saved as `<label>_<YYYYMMDD>.nc` (e.g. `trackbxg_w365_20260417.nc`). After a successful save it writes `output/posteriors/active_fit.json` (fit file, model, window dates, seasons, teams). `late_game.py`, `score_state.py` and `predict_game.py` read that file (`src/models/active_fit.py`), so all three use the same fit and the same games; `predict_game.py` warns if `late_game.json` or `score_state.json` were measured on a different fit. `--model goals --window 730` still fits v1 for comparison. The old `hier_20242025_20252026.nc` is left in place for the Phase 5/6 notebooks.

**Settings kept:** weekly refits, 5,000 score-state sims per game in the backtest, seed 42 (+ refit number), 1,000 posterior draws per prediction.

**Notes:**

- **Missing-xG check.** `fit_hierarchical.py` stops if any loaded regular-season game in the window (or after it, with the default end) has no rows in `shot_xg`, and prints the `score_xg --season` command to run. Without it, unscored games drop out of the game table and the fit quietly uses older data and goalie ratings. `--skip-xg-check` overrides.
- The latest `backtest.py` returns early ("All games already predicted; nothing to fit.") when a results file is complete, instead of redoing the pre-season fit.
- The notebook discovers model files by label, so any new backtest (new window or half-life) appears without code changes.
- Calibration slope: logistic regression of the outcome on logit(p). For a model that predicts the same value every game (home rate) it is undefined.

## Phase 8 — PCE for Sobol (Week 9+) — DONE (analysis A)

**Question (analysis A):** across the league, what makes one game's prediction different from another's? Team offense, team defense, the starting goalies, or the league-wide terms? Analysis B (which posterior uncertainties drive one matchup's 90% interval) is in v2 (see *v2.0c*).

**Files:**

- `src/models/stochastic/pce.py` — port of the soil driver's PCE code: `theta_to_xi`, `xi_to_theta`, `total_order_indices`, `legendre_eval`, `build_design_matrix`, `lhs_samples`, `fit_pce`, `pce_predict`, `loocv_q2`, `holdout_test`, `sobol_from_pce`, `saltelli_sobol`, `build_pce_surrogate`, `plot_pce_validation`, `plot_sobol`. Added: `sobol_groups`, `ishigami` / `ishigami_exact` (known-answer test), a vectorized option in `build_pce_surrogate`, a faster `build_design_matrix` (same result), and a recursive `total_order_indices` (same set and order; the soil version loops over $(P+1)^d \approx 10$ million combinations for $d = 10$).
- `src/analysis/sensitivity.py` — `INPUT_NAMES`, `GROUPS`, `input_specs(draws, gt, late)`, `game_qoi(theta, late)` (exact game model for arrays of inputs), `truth_predictor`, `sd_contributions`
- `src/pipelines/build_pce.py` — `py -m src.pipelines.build_pce [--degree 4] [--n-train N] [--saltelli 16384] [--overwrite]` → `output/models/pce_<fit stem>.pkl`, `_summary.json`, `_validation.png`, `_sobol.png` (about 40 s)
- `notebooks/phase8_check.ipynb` — eleven sections (below); the degree scan takes a few minutes

**Truth model:** the exact game model (`outcome_probs`), about 60,000 games per second, so the PCE design and the Saltelli check both run on the real model. PCE is not needed for speed here; it earns its keep by giving Sobol' indices analytically from its coefficients (see *Why PCE Earns Its Keep*).

**Inputs (10), each uniform with the real mean and sd** (range = mean ± √3 sd, Legendre basis):

| Input | Varies over | Source |
|---|---|---|
| `off_home`, `off_away` | the 32 teams | sd of the teams' posterior-mean offense ratings (active fit) |

| Input | Varies over | Source |
|---|---|---|
| `def_home`, `def_away` | the 32 teams | same, defense |
| `goalie_home`, `goalie_away` | starters in the fit window | sd of the starters' $\gamma$ |
| `alpha0`, `alpha_home`, `rho` | posterior uncertainty (same in every game) | posterior mean and sd |
| `ot_factor` | its uncertainty | `late_game.json` 90% range ÷ 3.29 |

Sobol' indices are shares of variance, so how much each input varies is the main modeling choice; variance matching keeps every input at its real spread. Section 10 repeats the analysis with full observed ranges (worst to best team, 1st–99th percentile goalie) to check that the conclusion doesn't depend on that choice.

**QoIs:** P(home win), P(tied after regulation), expected regulation goals (both teams).

**PCE settings:** total degree $P = 4$ (1,001 terms), 3 training points per term (3,003 Latin hypercube points), 2,000 hold-out points, seed 42. Grouped indices: team ratings / starting goalies / league terms.

**Checks (`phase8_check.ipynb`):**

| Section | Check |
|---|---|
| 2. Known answer | Ishigami function: PCE ($P = 10$) and Saltelli reproduce the exact indices, including $S_3 = 0$, $S_{T,3} = 0.244$ (test run: PCE within 0.0003) |
| 4. Truth model | Vectorized `game_qoi` = `outcome_probs` game by game; average matchup favors home a little; directions right (home offense up → P up; away goalie up → P down) |
| 5. Degree | Hold-out error for $P = 2..5$; indices stable from $P = 3$ |
| 6. Accuracy | Saved PCE: hold-out RMSE < 1% of the output sd; built on the active fit |
| 7. Main result | Sobol' $S_i$, $S_{T,i}$ per QoI; PCE vs Saltelli on the exact model (16,384 × 12 runs) within 0.01 |
| 8. Groups | Team ratings vs goalies vs league terms |
| 9. Units | Main-effect curves: P(home win) as each input moves across its range |
| 10. Robustness | Largest group unchanged and group shares move < 0.10 with full-range inputs |

**Pipeline test (fake database, 180 games; not real results):** PCE hold-out $R^2 \geq 0.99999$ for all three QoIs at $P = 4$ (RMSE 1.7e-4 on P(home win)); indices identical to 3 decimals from $P = 3$ to $5$; PCE and Saltelli agree within 0.0005; first-order sums 0.93–0.995 (little interaction). (The fake fit's shares are not meaningful; real results below.)

**Results (active v1.1 fit, 365-day window; all notebook checks PASS).** PCE and Saltelli on the exact model agree within 0.0003 on every index.

| Input | P(home win) $S_i$ | sd alone (prob.) | P(tied after reg.) $S_i$ / $S_{T,i}$ | Reg. goals $S_i$ |
|---|---|---|---|---|
| `off_home` | 0.248 | 0.032 | 0.122 / 0.154 | 0.221 |
| `def_away` | 0.238 | 0.031 | 0.118 / 0.148 | 0.213 |
| `off_away` | 0.234 | 0.031 | 0.040 / 0.070 | 0.199 |
| `def_home` | 0.225 | 0.030 | 0.038 / 0.068 | 0.192 |
| `alpha_home` | 0.031 | 0.011 | 0.015 / 0.020 | 0.027 |
| `goalie_away` | 0.012 | 0.007 | 0.006 / 0.008 | 0.011 |
| `goalie_home` | 0.011 | 0.007 | 0.002 / 0.004 | 0.010 |

| Input | P(home win) $S_i$ | sd alone (prob.) | P(tied after reg.) $S_i$ / $S_{T,i}$ | Reg. goals $S_i$ |
|---|---|---|---|---|
| `alpha0` | 0.000 | 0.000 | 0.044 / 0.044 | 0.125 |
| `rho` | 0.000 | 0.000 | **0.549** / 0.550 | 0.000 |
| `ot_factor` | 0.000 | 0.000 | 0.000 | 0.000 |
| **Output mean / sd** | 0.524 / 0.064 | | 0.259 / 0.016 | 5.64 / 0.44 |
| Sum of first-order | 0.999 | | 0.934 | 0.997 |

Grouped (first order; total in brackets where different):

| Group | P(home win) | P(tied after reg.) | Reg. goals |
|---|---|---|---|
| Team ratings | **0.946** | 0.375 (0.384) | **0.826** |
| Starting goalies | 0.023 | 0.008 (0.011) | 0.020 |
| League terms | 0.031 | **0.608** (0.614) | 0.152 |

Robustness (full observed ranges instead of variance-matched): P(home win) team ratings 0.942, goalies 0.035, league terms 0.023; ties 0.458 / 0.014 / 0.514; goals 0.851 / 0.032 / 0.116. Largest group unchanged everywhere; largest move 0.095 (ties).

**What Phase 8 found:**

1. **Team ratings decide who wins: 95% of the variation in P(home win) between games.** P(home win) has sd 0.064 across matchups (typical games 46–59%, a narrow league). Offense and defense matter equally (0.48 vs 0.46); each team input alone moves P(win) by about ±3 points (1 sd). Inputs pair as the model's structure says: home offense ≈ away defense (both set the home scoring rate), away offense ≈ home defense.
2. **Almost no interactions** (first-order sum 0.999): P(win) is close to additive in the ratings. Each input's effect barely depends on the others.
3. **Starting goalies: 2%** (about ±0.7 points each). Small because the Phase 3 goalie ratings vary little between starters (strongly shrunk, k = 800; see the input table in notebook section 3) next to team ratings, which matches Phase 3's weak year-to-year goalie signal. This is how much goalies matter *in the model*, not necessarily in hockey: if the rating is over-shrunk, real goalie effects are larger. The v2.0b wrong-starter test and the v2.1 goalie effect are the checks.
4. **Of the league terms, only home ice matters for the winner** (3%, about ±1.1 points from its posterior uncertainty). Scoring level, tie inflation and the OT factor have essentially zero effect on P(win). Pinning down home ice (more seasons, season-varying home ice in v2.3) is the only league-term improvement that would move P(win).
5. **Ties are driven by tie-inflation uncertainty** ($\rho$: 55%), then the home side's scoring rate. The league-term indices here measure *our uncertainty* about one shared number, so a sharper $\rho$ (more data, or the score-state simulator's tie mechanism in the exact model) would tighten OT and standings predictions; it won't change who wins. Ties are the only output with real interactions (7%): a tie depends on how *balanced* the two scoring rates are, which involves both teams together.
6. **Totals:** team ratings 83%, league scoring level 12%. Season-to-season drift in scoring ($\alpha_0$) is the next lever after ratings, consistent with Phase 7 (totals limited by scoring drift).

**What this means for v2:** improving team ratings is where nearly all the leverage on P(win) is, which supports v2.0 (time-varying ratings) as the first model item. Goalie and home-ice work can at most move P(win) a few percent of its variance unless the goalie ratings are too shrunk (test before building v2.1's goalie effect).

**Run order:** rebuild order (`fit_hierarchical` → `late_game` → `score_state`), then `py -m src.pipelines.build_pce`, then the notebook. Rebuild after each new fit; the indices shift slightly as ratings change.

**Expected from the model's structure:** P(win) depends on the inputs only through the two scoring rates, $\lambda_h$ (home offense − away defense − away goalie + home ice) and $\lambda_a$. So offense and defense shares split by how much each varies across teams, goalies get a share set by their (small) spread, and $\alpha_0$, $\rho$ and the OT factor should barely matter for P(win) but dominate ties and totals. Because the model only sees offense minus the opponent's defense, "offense vs defense" credit depends on how ratings vary across teams, not on a separate effect.

## v2 — Planned Improvements

Everything in this section comes **after Phase 7**. The baseline each item must beat is Track B v1.1 (1-year window): held-out log loss 0.6773, slope 0.99, tied with rolling xG (0.6754). Each item is kept only if it improves held-out log loss in the Phase 7 walk-forward backtest (`backtest_season.py` + `phase7_check.ipynb`).

| Item | What it adds | Main payoff |
|---|---|---|
| **v2.0 Time-varying team ratings (first)** | Ratings that drift over time at an estimated rate (Track B option C), on top of v1.1 | Phase 7's remaining gap: react to teams that change without losing confidence in teams that don't. Target: beat rolling xG, and make blending with it stop helping |
| **v2.1 Hierarchical xG layer** | Shooter, goalie, rink and season × shot-type effects on top of LightGBM xG | Finishing talent, a shrunk xG-based goalie rating, fixes for arena bias and redirect drift |
| **v2.2 Lineup model** | Per-skater offensive/defensive effects (Bayesian RAPM), aggregated over tonight's lineup | Predictions that react to injuries, scratches and trades |
| **v2.0c Phase 8 analysis B** | Sensitivity of one matchup's P(win) interval to each posterior uncertainty (correlated inputs: Shapley effects) | Explains what makes a prediction uncertain; points to which ratings need more data |
| **v2.0b Phase 7 follow-ups** | Market comparison, subset results, wrong-starter cost, home-win share, forward test on 2026-27 | Measures how far v1.1 is from the market and where it is weakest, to aim the model items |
| **v2.3 Other candidates** | In-game win probability, rest/travel effects, team- and season-specific home ice, better OT/SO, etc. (xG in Track B and score-state simulation are done) | Smaller calibration gains |

### v2.0c — Phase 8 Analysis B: What Drives One Matchup's Uncertainty

Phase 8 did analysis A (what makes games differ across the league). Analysis B asks, for one game: which posterior uncertainties make its 90% P(win) interval wide (e.g. "COL–DAL's interval comes mostly from DAL's defense rating")? Inputs are that game's posterior draws (team ratings, home ice, $\rho$; goalie uncertainty once v2.1 makes it a draw instead of a fixed value). Those inputs are **correlated** (offense with defense, $\alpha_0$ with the ratings, $r = -0.81$), and Sobol' indices assume independent inputs, so the options are: decorrelate first (Rosenblatt / Nataf transform, or a PCE on whitened inputs; indices then belong to the transformed inputs), or Shapley effects, which share variance fairly among correlated inputs. Start with Shapley effects on the exact model (cheap); report next to the posterior-correlation matrix.

### v2.0b — Phase 7 Follow-Ups (validation, not model changes)

Left open when Phase 7 closed. None changes the model; each tells v2 where to aim. All use the Phase 7 backtest files (`output/backtests/*_scored.csv`) and `phase7_check.ipynb` unless noted.

1. **Market comparison (most important).** The only test against the real benchmark: v1.1 ties rolling xG, but its gap to the betting market is unknown. Needs historical closing moneylines for 2024-25 and 2025-26 saved as `data/external/closing_lines.csv` (columns `date, home, away, home_ml, away_ml`; team abbreviations as in the database). Notebook section 10 then removes the margin (divide each implied probability by their sum, about 1.04–1.05) and scores the market like the models: log loss, Brier, calibration, paired difference vs v1.1. Realistic target: behind the market; each v2 model item is then measured by how much of that gap it closes. Also compare by subset to see where model and market disagree.
2. **Results by subset.** Log loss, calibration and paired differences vs rolling xG for: the first month of the season (ratings carried over the summer), back-to-backs and rest days, games started by backup goalies, and games after trades. Shows which v2 item would help most (v2.0 for early season and trades, rest covariates for back-to-backs, v2.1 goalie effect for backups).
3. **Cost of a wrong default starter.** The backtest used actual starters. Live predictions guess the starter (most starts in the last 10 games) unless `--home-goalie` / `--away-goalie` are given. Rerun the backtest predictions with the default starter instead of the actual one and compare log loss; the difference is what a wrong guess costs, and whether waiting for the confirmed starter is worth it.
4. **Home-win share on held-out games.** In-sample the model gave 52.9–53.4% vs 54.2% actual. Check mean predicted vs actual home-win share per season in the backtest; if it is low out of sample too, look at the home-ice prior or season-varying

home ice (v2.3).

5. **Forward test on 2026-27 — RUNNING (started 2026-10-24).** Nothing was tuned on these games, so it is the cleanest check of the live setup (weekly refits, guessed starters, real-time data).
   - `src/pipelines/predict_day.py`: `py -m src.pipelines.predict_day [--date D] [--days N] [--goalie TEAM=NAME|ID ...] [--list-goalies TEAM ...] [--replace]` predicts every regular-season game on the date(s) from the `schedule` table and appends to `output/forward/predictions_<season>.csv` (P(win) with 90% range, regulation/OT split, rates, fair odds, totals, puck lines, total-goals distribution, starters and whether they were given or guessed, historical home rate as a baseline, fit file, timestamp). It refuses games already loaded as played and dates inside the active fit's window. Run it in the morning: the schedule has dates, not puck-drop times.
   - `src/pipelines/score_forward.py`: `py -m src.pipelines.score_forward [--season S]` scores, per game, the last prediction made on or before the game date: log loss, Brier, accuracy, calibration slope (90% ranges) vs the historical home rate and the Phase 7 backtest (0.6773); calibration table; by week; score outputs; and **how often the default starter was right** (with log loss on wrong-guess games), which also covers item 3 going forward. Writes `output/forward/scored_<season>.csv`.
   - Expect a wide range for months: about ±0.03 in log loss after 300 games; differences of 0.005 need 1,000+ games.
   - Test on the fake database (15 games): skips, `--replace` with given starters, leak guards and scoring all worked.
   - **Game cards.** `predict_day` draws a pregame card for every game it logs, from the same numbers it logs (no second model run, so card and log always match); `--no-cards` turns it off, and a card that fails never stops the logging. Postgame cards: `py -m src.pipelines.make_cards postgame --pending` draws one for every logged game that is now loaded and scored (skips cards already drawn; `--redo` redraws), `--date D` for every loaded game on a date (logged or not), or give game ids. The postgame card shows the pregame P(win) the forward test scores (last prediction made on or before game day). Its "deserve to win" meter replays the game's shots 10,000 times, each scoring with its xG (`src/analysis/deserved.py`; tied replays go to the team with the larger xG share, the Phase 6 OT rule). `scripts/demo_cards.py` draws both cards on example data; `scripts/inspect_for_cards.py` prints the tables and signatures the cards use.

## v2.0 — Time-Varying Team Ratings

**Why first.** Phase 7 showed what Track B v1.1 is missing: recent form. Rolling xG reacts quickly (last 25 games, no shrinkage) and ties v1.1; averaging the two still helps by about 0.003. A recency half-life was tried in Phase 7 and is the wrong tool: it forgets every team at the same rate and makes the model less sure about all of them (slopes 1.5–1.6).

**Model.** v1.1 with each team's shot-quality ratings drifting as a random walk, by week within a season and with a larger step across the off-season:

$$\mu^{xG}_{\text{off},t,w} = \mu^{xG}_{\text{off},t,w-1} + \varepsilon_{t,w}, \qquad \varepsilon_{t,w} \sim \mathcal{N}(0, \tau^2_{\text{off}}), \qquad \text{off-season step} \sim \mathcal{N}(0, \tau^2_{\text{season}})$$

(same for defense). $\tau$ is estimated, so the data decide how fast teams change; a team that stays steady keeps a confident rating. Non-centered steps from the start. Weekly steps keep the parameter count manageable (about 32 × 2 × 26 per season) and match the weekly refits. Finishing effects $\phi$ stay static and tightly pooled. A middle step if sampling is hard: one rating per team per season, pulled toward the previous season's (option B).

**Checks.** Synthetic recovery with known drifting ratings (does it recover $\tau$ and the paths?); held-out backtest vs v1.1 and rolling xG; calibration slope; whether blending with rolling xG still helps (if it does, recent form isn't captured yet). It can use a longer window (2+ seasons), since old games are no longer treated as current.

## v2.1 — Hierarchical xG Layer

**Why a layer, not a fully Bayesian xG model.** LightGBM already pins down the shot-geometry function: with 562k shots, uncertainty in the distance/angle/shot-type curve is negligible next to game randomness and team-strength uncertainty. A fully Bayesian xG model would be slow on 562k shots and would need its nonlinear interactions written by hand, likely losing AUC. What LightGBM *can't* do is tell shooters, goalies and rinks apart, or let shot-type conversion drift by season. A small hierarchical model on top of it adds exactly that.

**Model.** LightGBM's prediction enters as a fixed offset; only the random effects are estimated:

$$
\begin{aligned}
\operatorname{logit} p_i &= \operatorname{logit}(xG_i^{\text{lgbm}}) + \beta_{s[i]} - \gamma_{g[i]} + \delta_{r[i]} + \varepsilon_{k[i]} \\
\beta_s &\sim \mathcal{N}(0, \sigma_\beta^2) \quad \text{shooter finishing talent} \\
\gamma_g &\sim \mathcal{N}(0, \sigma_\gamma^2) \quad \text{goalie skill (positive = better, same sign as Track B)} \\
\delta_r &\sim \mathcal{N}(0, \sigma_\delta^2) \quad \text{rink / arena recording effect} \\
\varepsilon_k &\sim \mathcal{N}(0, \sigma_\varepsilon^2) \quad \text{season} \times \text{shot-type group} \\
y_i &\sim \text{Bernoulli}(p_i)
\end{aligned}
$$

Half-normal hyperpriors on the four $\sigma$'s. Partial pooling shrinks low-volume shooters and goalies toward zero; high-volume ones keep their own estimate.

**What each term addresses:**

- $\beta$ — finishing talent. Feeds player point distributions later.
- $\gamma$ — a shrunk, xG-adjusted goalie rating. Replaces the Phase 3 conjugate rating if it backtests better.
- $\delta$ — the arena recording bias from Open Questions. A logit intercept per rink is an approximation: the real bias is in recorded *location*, so a distance-adjustment version is a later refinement. Home teams play 41 games in their own rink, so $\delta$ is partly confounded with home shooters and goalies; fitting all terms jointly is what separates them.
- $\varepsilon$ — the redirect location drift. Lets conversion by shot type move season to season instead of relying only on retraining.

**Inputs — the one hard rule.** The offset must come from the walk-forward `shot_xg` table (Phase 3), never from in-sample LightGBM predictions. In-sample xG is overfit, its residuals look too small, and the random effects would be shrunk toward zero.

**Fitting.**

- NumPyro NUTS (or PyMC with the NumPyro backend). Plain PyMC on 562k Bernoulli rows is slow.
- Non-centered parameterization for every random effect from the start.
- Parameter count is a few thousand (shooters dominate) — fast once the offset does the heavy lifting.

**Files:**

- `src/models/stochastic/xg_hierarchical.py` — model definition.
- `src/pipelines/fit_xg_layer.py` — load `shot_xg` + IDs, fit, save `output/posteriors/xg_layer_<season>.nc`.
- Adjusted per-shot xG (posterior mean and sd) written to a `shot_xg_v2` table; `shot_xg` stays untouched.

**Validation — keep the layer only if it earns it:**

1. **Synthetic recovery.** Simulate shots on top of real offsets with known $\beta, \gamma, \delta, \varepsilon$; check recovery, same discipline as the soil project.
2. **Held-out shot log loss.** Fit on seasons up to $t-1$, score season $t$; must beat the LightGBM offset alone.
3. **Posterior predictive checks.** Goals per shooter, per goalie and per rink against their predictive intervals.
4. **Sanity of the spreads.** $\sigma_\beta$ and $\sigma_\gamma$ should be small (finishing and goaltending talent are real but modest on the logit scale).
5. **Game-level payoff.** Swap $\gamma$ into the game model in place of the Phase 3 goalie rating and rerun the Phase 7 backtest. Keep whichever gives better game log loss.

**Goalie handoff (resolved).** Track B has no estimated goalie term; it takes goalie offsets as fixed inputs from Phase 3. To use the v2.1 goalie effect, swap it in as that input: one $\gamma$ posterior draw per Track B posterior draw, which also carries the goalie uncertainty Track B v1 ignores. Goalie skill still enters only once.

### v2.2 — Lineup (Player-Level) Model

**Idea.** The v1 game model rates *teams*. It can't tell when a top player is injured, scratched or traded. The lineup model rates each *skater's* effect on xG for and against, then builds tonight's team strength from tonight's dressed lineup. It is essentially Bayesian RAPM, fit with the same hierarchical/NUTS machinery as Track B.

**Data unit: the stint.** A stint is a stretch of time during which the set of players on the ice doesn't change; any line change by either team ends it. `stints.py` builds them by overlaying the 12 on-ice players' intervals from `shifts` and cutting at every change. Each stint records duration, the 10 skaters and 2 goalies, strength, score state, zone start, and shots and summed xG for each side. Expect a few hundred stints per game, on the order of a couple million over five seasons.

**Model (sketch).** For stint $j$ with duration $t_j$:

$$\text{rate}_{\text{home},j} = \alpha_0 + \alpha_{\text{home}} + \sum_{p \in \text{home skaters}} o_p - \sum_{q \in \text{away skaters}} d_q + (\text{score, zone terms})$$

and the mirror image for the away side. Each skater gets an offensive effect $o_p$ (raises his team's xG rate) and a defensive effect $d_p$ (suppresses the opponent's). Partial pooling learns the prior widths instead of tuning a ridge penalty, and the joint posterior shows when linemates can't be told apart (their effects come out negatively correlated).

**From ratings to a game prediction.**

1. Take tonight's dressed lineup (known about an hour before puck drop).
2. Project each player's ice time from **past games only**.
3. Team xGF/60 and xGA/60 ≈ ice-time-weighted sums of the dressed players' $o_p$ and $d_p$.
4. Convert xG to goals with finishing and goaltending effects (tonight's starting goalie; later the v2.1 shooter $\beta$ and goalie $\gamma$).
5. Feed $\lambda_{\text{home}}, \lambda_{\text{away}}$ into the same Phase 6 game model and Monte Carlo. Nothing downstream changes.

**Choices and tentative defaults.** None of these need deciding until Phase 7 is done; the defaults are starting points, not decisions.

*Before starting*

| # | Choice | Options | Tentative default |
|---|---|---|---|
| 1 | Order within v2 | Lineup model first, xG layer first, or both | xG layer first — smaller, and its $\beta, \gamma$ feed step 4 |
| 2 | Ridge RAPM reference | Yes / no | Yes — fit `deterministic/rapm.py` with scikit-learn first and check the Bayesian means against it |

*Model design*

| # | Choice | Options | Tentative default |
|---|---|---|---|
| 3 | Likelihood | Weighted Gaussian on xG/60; Tweedie on summed xG; two-part (shot counts + shot quality) | Weighted Gaussian (variance ∝ 1/duration); upgrade later |
| 4 | Scale of player effects | Linear on the xG rate, or log scale (effects multiply) | Linear — comparable to public RAPM |
| 5 | Game situations | 5v5 only, or separate PP/PK player effects | 5v5 only; special teams stay team-level |
| 6 | Stint covariates | Score state, zone start, home ice, back-to-backs | Score state, zone start, home ice |
| 7 | Prior structure | One shared $\sigma$; separate forward/defense $\sigma$; prior centered on last season | Separate F/D $\sigma$; no carryover in the first version |
| 8 | Data window | Rolling 2–3 seasons, or all 5 with recency weights | Rolling 2 seasons |
| 9 | Goalies in the stint model | In or out | Out — goalies affect whether shots go in, not shot quality; they enter at step 4 |
| 10 | Fitting method | NumPyro NUTS, or variational / Laplace approximation | NumPyro NUTS, sparse design matrix; approximate only if too slow |

*Making a prediction*

| # | Choice | Options | Tentative default |
|---|---|---|---|
| 11 | Ice-time projection | Last-$N$-games average share, or line-based deployment | Last 10 games, rescaled so the dressed lineup sums to a full game |
| 12 | Lineup source in the backtest | Actual dressed lineup vs. projected lineup | Actual dressed lineup (known pregame) with **projected** TOI — never that game's real TOI |

| # | Choice | Options | Tentative default |
|---|---|---|---|
| 13 | xG → goals | League-average finishing + Phase 3 goalie rating, or v2.1 $\beta, \gamma$ | League average + Phase 3 goalie first; swap in v2.1 later |

*Evaluation*

| # | Choice | Options | Tentative default |
|---|---|---|---|
| 14 | What counts as winning | Better overall game log loss, or better on games with major lineup changes | Score both. Fix the "major change" threshold in advance (e.g. a top-6 F or top-4 D out) |

**Validation.** Same Phase 7 walk-forward harness as the team model: game log loss and Brier, overall and on the lineup-change subset. Plus synthetic recovery on simulated stints with known player effects. If the lineup model only wins on lineup-change games, consider a cheaper hybrid: the team model adjusted for known missing players.

**Files.** `src/processing/stints.py`, `src/models/stochastic/lineup_rapm.py`, `src/pipelines/fit_lineup.py`, posteriors to `output/posteriors/lineup_<window>.nc`.

### v2.3 — Other Candidate Improvements

Smaller items already listed under *Open Questions*, gathered here so v2 has one list:

- **Score effects, rest, travel, back-to-backs** as game-model covariates.
- **OT/SO model** weighted by goalie quality instead of a flat 50/50.
- **Goal dependence:** done in v1 for ties (Track B tie inflation, $\rho = 0.51$). Dixon-Coles or a bivariate model remain alternatives if Phase 7 shows margin-specific miscalibration.
- **Score-state simulation: built in Phase 6** for pre-game score outputs. Natural extensions: **in-game win probability** (start the simulator from the current score, time and goalie state), period-specific score effects, and simulating the 11% of empty-net goals from delayed penalties and early pulls.
- **xG in Track B: done in Phase 7** (Track B v1.1, xG as a second observed quantity sharing the team effects).
- **Arena bias, location version:** adjust recorded shot distance per rink, refining the v2.1 rink intercept.
- **Rebound and rush windows:** test 2/3/4 s by retraining xG and comparing validation log loss.
- **Player-level point distributions:** built on the v2.2 player effects plus the v2.1 shooter finishing effects.
- **Team-specific home ice:** $\alpha_{\text{home}} + \eta_t$, with $\eta_t \sim \mathcal{N}(0, \sigma_\eta^2)$ partially pooled toward the league value (same idea as the goalie shrinkage). Candidates are rinks with real reasons to differ (altitude in Colorado, travel to Winnipeg or Vancouver). Published work generally finds most team-to-team spread in home advantage is noise, so expect $\sigma_\eta$ to come out small and most teams to land near the league value.
- **Season-varying home ice:** $\alpha_{\text{home},s}$ per season, either partially pooled or a slow random walk, because home advantage has drifted down over time across sports. League value on 2021-22 to 2025-26: $\eta = 0.0715$ (about 7.4% more scoring at home, uncertainty about ±0.010).
- Both home-ice items add parameters (32 for teams, 1 per season), so each must improve held-out log loss in the Phase 7 backtest to be kept.
- **Time-varying team ratings:** moved up to **v2.0** after Phase 7.

---

## Key Conceptual Notes

### Cheap vs Honest Uncertainty Propagation

Both versions produce joint vector samples. The difference is *where the correlations live*.

**Independent marginal sampling (cheap, fragmented).** Each component fit independently, sampled independently. Joint distribution is the product of marginals → diagonal covariance. Misses correlations between parameters that exist in the data (e.g., team offense and defense are correlated; team strength and goalie skill are negatively correlated for bad-team-good-goalie scenarios).

**Joint hierarchical posterior (honest).** All parameters fit jointly in a single hierarchical model. Sample the whole vector at once → full covariance, possibly non-Gaussian. This is what Track B produces.

Cheap version typically miscalibrates the tails. Won't catastrophically break win probabilities but will produce wrong probabilities for extreme outcomes.

### Why PCE Earns Its Keep for Hockey

For the soil project, PCE was essential because the truth model was ~1 s/eval. For the hockey team-level game model, evaluation is in microseconds — direct MCMC on the truth model is viable, surrogate not strictly needed for inference.

PCE earns its keep in hockey for **sensitivity analysis**: analytical Sobol indices directly from PCE coefficients via the variance-decomposition identity

$$\mathrm{Var}[\hat{Y}] = \sum_{|\alpha|>0} c_\alpha^2 \mathbb{E}[\Psi_\alpha^2]$$

This gives a clean decomposition of outcome variance into contributions from each input (team xGF/60, team xGA/60, goalie GSAx, etc.) — without running Saltelli on millions of samples.

If you later upgrade to a possession-by-possession simulator that's slow to run, PCE returns as an inference-speed tool.

### Sources of Uncertainty (Ranked by Sobol Later)

Three categories injected at different layers:

1. **Aleatoric (irreducible).** Game randomness — Poisson sampling in the game model. Even with perfect knowledge of $\lambda$, the puck still bounces.
2. **Parameter uncertainty.** Posterior width on team/goalie/player parameters from finite-sample inference.
3. **Model misspecification.** The Poisson form, the independence assumption between home/away goals, the missing factors (rest, travel, injuries). Hardest to quantify; partially addressed by tie inflation (Phase 5), posterior predictive checks and held-out validation.

Sobol analysis at the end ranks which of these matter most for which outcomes.

---

## Persistence & Reproducibility

- **xG model:** pickle under `output/models/`, named with `MODEL_VERSION` (`xg_lgbm_v1_*`), plus a `_metrics.json` with version, seed, settings, data cutoff and library versions. `XG_MODEL_PATH` in `config/settings.py` points to the active model.
- **Per-shot xG:** the `shot_xg` table, rebuilt by `score_xg.py`; each row records the model version and which model scored it.
- **Posterior samples:** ArviZ NetCDF under `output/posteriors/`. Both tracks write to the same format so downstream code is sampler-agnostic. Track A writes `toy_<A>_<B>.nc` (posterior + observed games, settings in `posterior.attrs`) plus `_meta.json` (seed, sampler settings, game ids, diagnostics, library versions) and trace/ACF plots. Like `.pkl` files, these stay out of git; rebuild with `fit_toy.py`. Track B writes `<label>_<YYYYMMDD>.nc` (e.g. `trackbxg_w365_20260417.nc`; label = model, window, half-life; date = window end; seasons, window dates and team list in `posterior.attrs`), `_meta.json` (priors, window, sampler settings, diagnostics, versions), `_ratings.csv` and `_trace.png`; rebuild with `fit_hierarchical.py`. `active_fit.json` names the fit the live pipeline uses. (Phase 5–6 fits are `hier_<first>_<last>.nc`.)
- **Late-game and score-state parameters:** `output/models/late_game.json` and `output/models/score_state.json`, each recording the fit file they were measured against; rebuild with `late_game.py` / `score_state.py` after every new fit.
- **Game cards:** `output/game_summaries/<date>/<away>_at_<home>_<pregame|postgame>.png` (pregame from `predict_day`, postgame from `make_cards`).
- **Forward test:** `output/forward/predictions_<season>.csv` (append-only log, one row per prediction, timestamped) and `scored_<season>.csv`.
- **Predictions:** `output/predictions/<date>_<away>_at_<home>.json` (`predict_game.py --save`): inputs, goalies used, outcome probabilities with intervals, likely scores, totals and puck lines.
- **PCE surrogate:** `output/models/pce_<fit stem>.pkl` (coefficients, multi-indices, input specs, Sobol' indices, hold-out set) and `_summary.json` (input ranges and sources, accuracy, single and grouped indices, Saltelli check); rebuild with `build_pce.py` after each new fit.
- **Backtest results:** `output/backtests/<label>_<season>.csv` (one row per held-out game: teams, goalies, P(win) with 90% range, regulation/OT probabilities, rates, score outputs, refit date, training games, divergences) and `_scored.csv` (plus

final score, how it ended, `home_won`); baselines in `baselines_<season>.csv` + `_info.json` (tuned Elo settings). Labels name the model, window and half-life (`trackbxg_w365`).

Reproducibility checklist for any fit:

- Random seed logged
- Data cutoff date logged
- Model version logged
- Software versions (PyMC, NumPy, LightGBM) logged in metadata

---

## Open Questions Deferred to "When We Get There"

These are real decisions but premature to make now:

- **OOP vs functional structure** for the inference layer. Decide once 2–3 modules are written and friction becomes obvious.
- **Whether to use `__init__.py` re-exports** for cleaner imports. Default to empty files; refactor later if imports get ugly.
- **OT/SO model.** Start with a flat 50/50 conditional on tied regulation. Upgrade to a goalie-quality-weighted shootout model later if it materially changes calibration.
- **Score effects, rest, travel, back-to-backs.** Real factors, but adding them prematurely complicates the base model. Document as v2 features.
- **Player-level point distributions.** Requires shift-level data and per-player rate modeling. Defer until team-level pipeline is calibrated; planned on top of the v2.2 lineup model.

**Data and model items found in Phases 1–3:**

- **Redirect location drift.** Each season, redirects are recorded closer to the net (share under 10 ft: 24.8% in 2021-22 → 38.2% in 2025-26) while their goal rate falls (9.4% → 7.5%). The model overrates 2025-26 redirects by about 24%, and this accounts for most of the +2.4% total bias. Recency weighting helps only slightly. Retrain every season. The v2.1 season × shot-type effect ($\varepsilon$) is the planned fix.

- **Arena recording bias.** Some rinks log shot locations closer or farther than others. MoneyPuck adjusts for it; this model does not. Planned fix: rink effect ($\delta$) in v2.1.

- **Timestamp artifacts.** The feed logs whole seconds, so an opponent's shot can appear 1–2 s before a shot at the other net. Affects about 0.5% of shots.

- **NST rush definition.** NST appears not to count an opponent's shot as a rush precursor. Unconfirmed at three events; a query-time filter gives NST parity if needed.

- **Rebound and rush windows.** 3 s and 4 s are NST conventions, not fitted. Test 2/3/4 s by retraining and comparing validation log loss.

- **Previous-event location for stoppage-type events** is missing for about 0.5% of shots.

- **Season-level xG offset.** Leave-one-season-out xG runs about 3% high in 2021-22 and 2022-23 and about 3% low in 2023-24 and 2024-25 (xG/goals 1.032, 1.031, 0.974, 0.969, 1.024). Normal luck would move this about 1%, so goal conversion is changing between seasons more than shot quality explains (goaltending trends, recording changes, play the features miss). Within-season team comparisons are unaffected. Goalie ratings now handle it (`season_adjust=True` rescales each season to its own league average). The Phase 6 game model still needs a per-season league-average term or the same rescaling for team xG.

- **Early-season team strength.** v1 rolling windows span seasons (the last 25 games, whatever season), so every team has an estimate on opening night but off-season roster changes are ignored. Refinement: pull early-season estimates toward the league average.

- **When to retrain the xG model.** Keep the live model fixed during a season, so it never scores games it trained on. After the season ends, add it, move `CURRENT_SEASON` / `LAST_SEASON` forward one year, and run `train_xg.py --final` and `score_xg.py`.

- **Rename `CURRENT_SEASON` to `TEST_SEASON`.** It is used as the xG test season, not the season being played, which is easy to confuse.
- **Goalie signal is weak.** Adjusted next-season correlation is 0.125 (±0.12), so goalies should move game predictions only modestly. Repeat the `k` test when a new season is added; the best value sat on a flat stretch (400–1,600).
- **Team strength ignores schedule strength.** The rolling window doesn't adjust for the quality of opponents faced. Track B's joint fit handles this; the v1 rolling estimate does not.
- **Per-game MoneyPuck comparison** is still to do (Phase 2 gap).
- **Goalie uncertainty ignored in Track B v1.** Goalie offsets are fixed at the Phase 3 posterior mean (sd about ±0.03). Fix planned with the v2.1 goalie effect (draws instead of a fixed value).
- **Which goalie counts in Track B.** The starter's rating is used for the whole game; games where the starter is pulled get the starter's value. Splitting by time in net is a refinement if it ever matters.

**Model items found in Phase 5:**

- **Score effects / under-dispersion (resolved in Phase 6).** Regulation goals without empty-netters are less spread than Poisson (0.92) because of score effects; empty-net goals are part of the same behavior. Track B unchanged; score outputs come from the Phase 6 score-state simulator.
- **Track B uses goals, not xG (resolved in Phase 7).** Rolling xG beat v1; Track B v1.1 adds xG and ties it.
- **Fit window (resolved in Phase 7).** One season beats two for static ratings (two seasons were too confident out of sample).
- **Wide team intervals.** Every team's 90% net-rating interval is about ±0.09; mid-table teams overlap heavily. Expected with 164 games each; this uncertainty is what Phase 6 should carry into win probabilities.
- **OT model (resolved in Phase 6).** Sudden death at 2.39× the regulation rates; shootout 50/50 (data agree, p = 0.94).

**Model items found in Phase 6:**

- **Calibration too cautious at the extremes** (in-sample). **Resolved in Phase 7:** did not hold out of sample; the two-season v1 was too confident (0.79), v1.1 one-season is calibrated (0.99).
- **Home win slightly low** in simulation (52.9–53.4% vs 54.2%), inside the 90% range in-sample; recheck in Phase 7.
- **Remaining final-score spread** (score-state 1.011 vs 0.979 actual). Partly rating uncertainty; the rest small unmodeled effects (period-specific score effects, the 11% of empty-net goals outside late pulls). Not tuned in-sample.
- **Empty-net "hidden ties" in the exact model** (23.5% vs 22.8% past regulation). Doesn't affect P(win); score outputs use the score-state simulator. If standings simulations use the exact model, send about 96% of its regulation ties to OT.
- **Default starting goalie** is the team's most frequent starter in its last 10 games; real starters are announced on game day. Phase 7 can measure how much a wrong default costs.
- **2026-27 season.** Not yet loaded; predictions for current games need it plus the rebuild order (fit → late_game → score_state).

**Model items found in Phase 7:**

- **Track B ties but doesn't beat rolling xG,** and averaging the two still helps (about 0.003). The missing piece is recent form handled properly → v2.0 time-varying ratings.
- **Recency half-lives make Track B too cautious** (slopes 1.5–1.6 at 60–90 days): fewer effective games → wider ratings → predictions pulled toward 50%. Not adopted.
- **Rolling xG and Elo are too confident** (slopes 0.83 and 0.64): they treat point estimates as exact.
- **Market comparison not run yet:** needs historical closing moneylines (`data/external/closing_lines.csv`).
- **Total goals** are limited by season-to-season drift in league scoring; a per-season scoring term (or time-varying $\alpha_0$, with v2.0) could help.
- **Divergences per refit** are logged in the backtest CSVs; per-fit R-hat is not yet logged.

---

## Decisions Made

For future reference, the decisions locked in during planning:

- **First shipping target:** single-game win probability.

- **xG approach:** train own model, validate against MoneyPuck.
- **Uncertainty propagation:** honest version (joint hierarchical Bayesian via PyMC NUTS).
- **Build strategy:** Track A (AM MCMC) and Track B (PyMC) in parallel. Track A as integration test and reference; Track B as production.
- **Uncertainty sources to track:** all three (aleatoric, parameter, model misspec); rank with Sobol later, no pre-ranking.
- **Folder structure:** add `analysis/`, `validation/`, `pipelines/` as new top-level under `src/`. `game_model.py` at top level of `models/`.

Decided during Phases 1–2:

- **Raw tables stay raw.** Derived flags live in `shot_context`; filters are applied at query time.
- **xG training set:** unblocked shots, `period <= 4`, empty nets dropped.
- **Season roles:** test = `CURRENT_SEASON`, validation = `LAST_SEASON` (both in `config/settings.py`); the test season is graded once per change, never used to choose settings.
- **Rush in the model:** feed `last_event_type` and `time_since_last_event`; keep `is_rush` for reporting.
- **Strength state:** decoded shooter-relative (skater difference, total skaters, own goalie pulled), not the raw code.
- **Shot types grouped** into `wrist-snap` and `redirect` because the labels changed between seasons.
- **Recency weighting on** (1.0 / 1.25 / 1.5 / 2.0).
- **No recalibration step:** the remaining bias is concentrated in redirects, so a uniform shift would worsen other shot types.
- **Trained `.pkl` files stay out of git;** rebuild with `train_xg.py`.

Decided during Phase 3:

- **Per-shot xG comes from leave-one-season-out models** for historical seasons and from the live model for new ones; stored in `shot_xg`.
- **Team strength:** rolling 25-game window spanning seasons, total xG ÷ total minutes, variance = game-to-game variance ÷ games. Estimates use only earlier games.
- **Goalie rating:** Gamma-Poisson on GA ÷ xGA, `k = 800`, season-adjusted, two-season lookback. Chosen by the next-season prediction test in `phase3_check.ipynb`.
- **Goalie GA excludes empty-net and shootout goals;** team `gf`/`ga` include empty-net goals, `gf_xg`/`ga_xg` exclude them (the goal models use the latter; see Phase 4).
- **Build Phase 3 on complete seasons;** load 2026-27 when live predictions are needed. Keep the live xG model fixed during a season.

Decided 2026-10-17:

- **LightGBM stays as the xG base.** Shooter, goalie, rink and season effects go in a v2 hierarchical layer that uses walk-forward LightGBM xG as a fixed offset, not in a fully Bayesian replacement.
- **Track B gets a global intercept** $\alpha_0 \sim \mathcal{N}(\log 3, 0.5^2)$ and non-centered random effects from the start.
- **Track A reports only identified quantities** ($d_A, d_B$), with $\alpha_0$ fixed to the league mean.
- **Track A vs Track B agreement is tested like-for-like** (matched no-pooling Track B on the same games); the pooled comparison is qualitative only.
- **Lineup (player-level) model is v2,** after Phase 7. Its choices are listed in v2.2 with tentative defaults, not decided.
- **One goalie rating.** The Phase 3 goalie rating is built as a swappable interface; the v2 goalie effect replaces it only if it wins the Phase 7 backtest.

Decided during Phase 4:

- **Goalies enter Track B as a fixed input** ($\gamma = -\log\theta$ from the Phase 3 rating of the starter, as of game date), not as an estimated term. Avoids confounding with team defense and rating goalies twice. Cost: goalie uncertainty is ignored in v1 (about ±0.03, small next to team effects); the v2.1 goalie effect fixes it.

- **Track B v1 fits static team ratings on a recent window** (option A, e.g. the last 1–2 seasons), refit as time moves forward in the Phase 7 backtest. Per-season (B) and random-walk (C) ratings are v2.3 candidates.

- **Goal-model target excludes empty-net goals** (`gf_xg` / `ga_xg`) for Track A, Track B and the game model. Realistic final scores (6v5 endgame) can be a small Phase 6 add-on if ever needed; win probability does not need it.

- **Home ice is in the model.** Track A fixes it at the league value $\eta = 0.0715$; Track B estimates one league-wide $\alpha_{\text{home}}$ (expect about 0.07 ± 0.01, a free check). Team- and season-specific versions are v2.3 candidates.
- **Fixed league terms are computed from data at fit time,** not hard-coded, so they update as seasons are added.
- **Track A samples $(d_A, d_B)$ directly** (two parameters), not the four unidentified team terms.
- **Track A is a verification tool, not a prediction input.** Its saved posteriors are the reference for Track B's matched check.
- **Every MCMC fit runs 4 chains from spread-out starts** and reports R-hat, ESS and acceptance; posterior files carry the observed data and model settings.

Decided during Phase 5:

- **Track B fits regulation goals** (periods 1–3, no empty-netters). OT and the shootout are simulated separately in Phase 6. Track A comparisons use the periods 1–4 goals Track A was fit on.
- **Tie-inflated Poisson likelihood** ($\rho$, prior $\log(1+\rho) \sim \mathcal{N}(0, 0.5^2)$) instead of independent Poisson, fit jointly with the ratings. `tie_inflation=False` keeps the plain model as a baseline.
- **Fit window `FIT_SEASONS` = 2024-25 + 2025-26,** static ratings, priors $\alpha_0 \sim \mathcal{N}(\log 3, 0.5^2)$, $\alpha_{\text{home}} \sim \mathcal{N}(0, 1)$, $\sigma \sim \text{HalfNormal}(0.3)$.
- **No model change for dispersion.** It comes mostly from excluding empty-net goals (section 11). Phase 6 adds an empty-net step for final-score outputs; score-state simulation stays an optional v2.3 item for in-game predictions.
- **The betting market is a Phase 7 competitor:** closing moneylines, margin removed, scored like the models.
- **Track B checks live in `phase5_check.ipynb`:** game table, leakage, synthetic recovery, matched Track A check, diagnostics, league terms, sanity vs goals and xG, posterior predictive, pooled vs Track A.
- **Saved posteriors are read into memory and closed** (`load_posterior()`), so open notebooks never lock files on Windows.

Decided during Phase 6:

- **P(win) from the exact model, score outputs from the score-state simulator.** Each does better at its job (section 4 vs section 9); the prediction header prints both win shares as a consistency check.
- **Uncertainty = spread of per-draw probabilities.** Each posterior draw gives an exact P(win); mean = prediction, 5th–95th percentile = 90% interval. Game randomness is inside each P(win).
- **OT:** sudden death at $k$ = 2.39 × regulation rates for 5 minutes; who scores follows the rates. **Shootout:** 50/50.
- **Score-state simulator:** 10-second slices, measured score-state factors, pull hazard, 6-on-5 and empty-net rates, one calibrated scale, **no tie inflation**.
- **No in-sample tuning of calibration.** Calibration issues are tested and fixed on held-out games in Phase 7.
- **Default goalie** = most starts in the team's last 10 games, unless given on the command line.
- **Rebuild order:** `fit_hierarchical` → `late_game` → `score_state`.

Decided during Phase 7:

- **Production uses v1.1, 365-day window** (same as the backtest). One record of the active fit (`active_fit.json`), read by every downstream script, instead of each script building the file name itself.
- **Adopted model: Track B v1.1, 1-year window, no recency half-life.** Ratings from xG + goals; statistically tied with rolling xG, best calibrated model tested (slope 0.99), no tuning knob.
- **xG goes inside Track B** as a second observed quantity (Gamma likelihood on regulation xG) sharing the team effects, with small tightly pooled finishing effects for goals beyond xG. Goalie offsets act on goals only.
- **One-year window over two** for static ratings.
- **Recency weighting rejected:** it tied rolling xG in 2025-26 but hurt 2024-25 and calibration.
- **No rolling-xG blend in production.** Kept as a benchmark only (no honest interval, ignores the starter, disagrees with the simulator). Blending should stop helping once v2.0 works.
- **Time-varying team ratings are the first v2 item.**

- **Backtest protocol:** walk-forward, weekly refits on a rolling date window, late-game and score-state parameters measured pre-season, actual starters rated as of game date, baselines tuned only on earlier seasons. Models compared on the same games with paired bootstrap ranges; a difference counts only if its 90% range excludes 0.
- **Market comparison stays in Phase 7's notebook** (section 10) and runs once closing-line data is available.

Decided during Phase 8:

- **Analysis A only** (sensitivity across matchups); analysis B (one matchup's posterior uncertainty, correlated inputs) moved to v2.0c.
- **Truth model = exact game model,** evaluated directly (fast); Saltelli on the exact model checks the PCE indices.
- **Inputs uniform and variance-matched** to their real spread (teams across the league, goalies across starters, league terms over their posterior); full-range inputs as a robustness check.
- **Same PCE method as the soil project** (Legendre, total degree, least squares on LHS, LOO $Q^2$, hold-out test), degree 4 by default; no Morris screening step, since 10 inputs are cheap to keep.

---

## Glossary

**xG** — Expected Goals. Per-shot probability of becoming a goal, summed over shots. Output of a classifier (usually gradient boosting) trained on shot features.

**RAPM** — Regularized Adjusted Plus-Minus. Ridge regression isolating player impact on shot/goal rates while controlling for teammates and opponents.

**GSAx** — Goals Saved Above Expected. $xGA - GA$ for goalies. Positive = better than average.

**PDO** — On-ice $SH\% + SV\%$. Mean-reverts to 100. Luck/regression indicator.

**Corsi / Fenwick** — Shot attempt differentials. Corsi includes blocks; Fenwick excludes them.

**Game Score** — Dom Luszczyszyn's weighted-stat composite (~1.0 = average game).

**PCE** — Polynomial Chaos Expansion. Spectral surrogate for expensive forward models. Orthogonal polynomial basis under input distribution.

**Sobol indices** — Variance-based sensitivity measures. $S_i$ = fraction of output variance attributable to input $i$ alone; $S_{T_i}$ = total contribution including interactions.

**NUTS** — No-U-Turn Sampler. HMC variant. Default sampler in modern PyMC.

**AM** — Adaptive Metropolis. Random-walk MH with covariance estimated from burn-in.

**ArviZ** — Python library for Bayesian model diagnostics and storage. NetCDF-based `InferenceData` is the standard format.

**Walk-forward backtest** — Validation strategy where, for each held-out game in chronological order, models are refit using only prior data. Avoids look-ahead bias.

**AUC** — Area under the ROC curve. The chance the model gives a random goal a higher xG than a random non-goal. 0.5 is guessing; public xG models land around 0.75–0.80.

**Log loss** — How far predicted probabilities are from what happened. Lower is better; a proper scoring rule.

**Calibration** — Whether predicted probabilities match observed rates: shots given 10% should score about 10% of the time.

**Gradient boosting** — Many small decision trees built in sequence, each correcting the previous ones' errors (LightGBM, XGBoost). Contrast with a random forest, which averages independently built trees.

**Early stopping** — Stop adding trees once the validation score stops improving, to avoid overfitting.

**Recency weighting** — Counting recent seasons more in training so the model follows current recording habits.

**Stint** — A stretch of game time during which the set of players on the ice does not change. The unit of data for RAPM.

**Tweedie distribution** — A compound Poisson–Gamma distribution: a point mass at zero plus a continuous positive part. Fits summed xG per stint.

**Offset** — A term added to a model's linear predictor with its coefficient fixed at 1, not estimated. In the v2 xG layer, logit(LightGBM xG) is the offset, so the Bayesian model only learns adjustments to it.

**Partial pooling** — In a hierarchical model, group effects (teams, shooters, goalies) share a common prior whose width is learned. Groups with little data are pulled toward the average; groups with lots of data keep their own estimate.

**Non-centered parameterization** — Writing a random effect as $\beta_s = \sigma_\beta z_s$ with $z_s \sim \mathcal{N}(0,1)$ instead of sampling $\beta_s \sim \mathcal{N}(0, \sigma_\beta^2)$ directly. Removes the "funnel" geometry that causes NUTS divergences when $\sigma$ is small.

**Leave-one-season-out** — Scoring each season with a model trained on all the other seasons, so no prediction comes from a model that saw that data.

**Conjugate prior** — A prior that, combined with the likelihood, gives a posterior of the same family, so the update is a simple formula. Gamma is conjugate to Poisson: Gamma(k, k) becomes Gamma(k + GA, k + xGA).

**Credible interval** — A range that contains the true value with a stated probability under the posterior (e.g. 90%). The Bayesian counterpart of a confidence interval.

**Shrinkage** — Pulling estimates from small samples toward the average. In the goalie rating, `k` sets how strongly.

**Posterior predictive check** — Simulating new data from the fitted model (using posterior draws) and comparing it with the real data on statistics that matter. A statistic outside the simulated range shows where the model's assumptions break.

**Tie inflation** — Raising the probability of tied scores above what independent scoring gives, with one parameter $\rho$ (tied scores × $(1+\rho)$, then renormalized).

**Divergence (NUTS)** — A warning that the sampler hit a region of the posterior it can't follow accurately (usually sharp curvature). Even a few mean the results may be biased; the fix is usually a reparameterization.

**BFMI** — Bayesian fraction of missing information. Measures how well NUTS moves between energy levels; below about 0.3 suggests poor exploration.

**ESS** — Effective sample size. How many independent draws the correlated MCMC draws are worth. A few hundred per parameter is enough for means and 90% intervals.

**Score effects** — Teams play differently depending on the score: trailing teams take more risks and generate more shots, leading teams defend. Measured here: trailing teams score at 1.07–1.11× their normal pace, leaders at about 0.93×.

**Closing line** — A sportsbook's final odds before the game starts, after all betting and news (lineups, goalies, injuries). The standard benchmark for a prediction model.

**Vig (bookmaker's margin)** — The sportsbook's built-in edge: the implied probabilities of both sides add up to more than 1 (about 1.04–1.05 for NHL moneylines). Divide each by the sum to get fair probabilities.

**Fair odds** — Odds with no bookmaker margin, from a probability: p ≥ 0.5 → −100·p/(1−p); p < 0.5 → +100·(1−p)/p. What `predict_game.py` prints.

**Puck line** — A bet on the margin: −1.5 means winning by 2 or more (empty-net goals and the OT/shootout +1 count).

**6-on-5 / pulled goalie** — A trailing team replaces its goalie with an extra skater late in the game; it scores more often (about 2.7× here) but can concede into an empty net.

**Pull hazard** — The chance per second that a trailing team pulls its goalie, given the margin and time left; measured from play-by-play.

**Calibration slope** — Fit the outcome against logit(predicted probability) with a logistic regression. Slope 1 = calibrated; below 1 = predictions too extreme (over-confident); above 1 = too close to 50% (too cautious).

**Elo** — A rating system that moves each team's number after every game by $k$ × (result − expected result). Tuned here on earlier seasons: $k$, home edge, and how much rating carries over the summer.

**Paired bootstrap** — Resampling whole games and scoring both models on the same resampled games, so the range reflects the difference between models rather than which games happened to be played.

**Recency half-life** — Weighting each game by $0.5^{\text{age}/\text{half-life}}$, so a game one half-life old counts half as much.

**RPS (ranked probability score)** — Brier score for an ordered outcome (total goals): compares the predicted and actual cumulative distributions. Lower is better.

**Proper scoring rules** — Loss functions for probabilistic predictions that are minimized in expectation by the true distribution. Log-loss, Brier, CRPS are proper. RMSE on probabilities is not.